

GitLab CI/CD Architecture for Multi-Repository Environments

A comprehensive DevOps strategy for managing upstream, downstream, and mirror deployments with environment-specific configurations

datalab@unimib.it

Project:	dve-sample-py
Document:	Python-Q2c-GitLab-CI-Environment-Config.pdf
URL:	https://gitlab.com/ub-dems-public/ds-labs/dve-sample-py/notes/howtos/
Date:	July 16, 2026

Keywords: GEN, GitLab-CI, DevOps, Python, Podman, Multi-Environment

Abstract

This document outlines a comprehensive GitLab CI/CD strategy for managing a multi-repository ecosystem. It addresses the complexities of coordinating continuous integration pipelines across a public upstream repository, a private downstream fork, and a public GitHub mirror. The architecture is designed for a Python application managed with uv, incorporating code validation, automated testing, and containerized delivery.

Central to the proposed methodology is the resolution of environment- dependent configuration management. The framework evaluates mechanisms for securely storing and accessing sensitive data, such as API credentials, strictly within private Git repositories. A structured comparison of Git submodules versus parallel repository checkouts is detailed to establish best practices for consistent data availability across local development and distributed CI/CD environments.

Furthermore, the specification defines requirements for robust inter- repository authorization and the seamless injection of pipeline variables. It outlines a parallelized container image build process utilizing Podman, featuring a failure-aware dependency graph. The finalized strategy synthesizes these technical constraints into concrete operative procedures for robust software deployment.

Contents

TOC	5
Q:1	6
Q:1 - GitLab CI CD Multi Repo Configuration Strategy	6
Role	6
Terminology	6
Context	6
Objective	7
Output Format	8
A:1 (Claude)	9
1. Shared/Centralized CI Templates	9
2. CI/CD Components (GitLab 16.9+)	9
3. Multi-Project Pipelines (trigger jobs)	9
4. Parent-Child Pipelines	10
5. Group-Level Shared Configuration	10
6. Repo Topology Considerations	10
Environment × Repository Combinations	10
1. Configuration Management Strategy	11
git submodule	11
Parallel checkout (sibling directory, relative path)	12
Recommendation	12
2. Multi-Environment Configuration Resolution	13
3. Inter-Repository Authorisation	14
Downstream → Upstream (merge requests and tags)	14
Downstream → Mirror (GitHub)	14
spec:inputs overrides at trigger time	15
4. Container Registry and Secrets Availability	15
5. Parallel Container Image Builds With Failure-Aware Dependencies	16
6. Synthesis	17
Additional Notes	18
References	18
A:1 (Gemini)	19
Pre-analysis: Repository and Environment Matrix	19
The Upstream Repository (Public)	19
The Downstream Repository (Private)	19
The Mirror Repository (Public GitHub)	19

Objective 1: Configuration Management Best Practices	20
Git Submodule vs. Parallel Checkout	20
Recommendation	20
Objective 2: GitLab Environment Support	20
Configuration Resolution Strategy	20
Environment Resolution Table	21
Objective 3: Inter-repository Authorisation	21
GitLab to GitLab Authorisation	21
GitLab to GitHub Authorisation (Mirroring)	21
Pipeline Trigger Overrides and <code>spec:inputs</code>	21
Objective 4: Public Container Registry and Secrets Availability	22
Secrets Storage and Retrieval Table	22
Objective 5: Parallel Container Image Builds	22
Job Graph Behaviour	22
Job Graph Sketch	23
Objective 6: DevOps Strategy Synthesis	23
Operative Procedures	23
Critical Points	23
Pros and Cons Summary	24
Additional Notes	24
References	24
A:1 (ChatGPT)	25
Executive Summary	25
1. Configuration Management	26
Goals	26
Option A — Git Submodule	26
Option B — Parallel Checkout (<i>Recommended</i>)	27
Recommendation	27
2. Multi-Environment Configuration Resolution	28
Resolution Strategy	28
Resolution Table	28
3. Inter-Repository Authorisation	29
GitLab → GitLab	29
GitLab → GitHub	30
<code>spec:inputs</code> Default Override	30
4. Secrets Strategy	31
Principles	31
Storage Matrix	31
5. Parallel Container Image Builds	32
Failure Behaviour	33
6. Overall DevOps Strategy	34
Repository Responsibilities	34
Recommended Operational Workflow	34
Critical Considerations	35

Additional Notes	35
References	36
A:1 (DeepSeek)	37
Preliminary Analysis: Environment × Repository Combinations	37
1. Configuration Management Best Practices	37
Constraint	37
The Two Approaches	37
Comparison	38
Recommendation	38
2. Multi-Environment Configuration Resolution	38
3. Inter-Repository Authorisation	39
a) GitLab-to-GitLab: Downstream Triggers Upstream	39
b) GitLab-to-GitHub: Mirror Push	40
4. Public Container Registry and Secrets Availability	40
5. Parallel Container Image Builds with Failure-Aware Dependency Chain	41
Job Graph Description	41
Sketch of <code>.gitlab-ci.yml</code>	41
6. DevOps Strategy Synthesis	43
Concrete Operative Procedures	43
Critical Points and Trade-offs	44
Comparison of Alternatives	44
References	44
Additional Notes	45
A:1 (Perplexity)	46
1) Configuration management	46
Submodule vs parallel checkout	46
2) Environment resolution	46
3) Cross-project authorization	47
<code>spec:inputs:</code> override behavior	47
4) Secrets and registry access	47
5) Parallel image builds	48
Job graph sketch	48
6) Operative procedures	49
Repository-specific nuances	50
Critical points	50
References	50
A:1 (Vibe)	52
1. Configuration Management Best Practices	52
Comparison: <code>git submodule</code> vs. Parallel Checkout of a Configuration Repository	52
Final Recommendation	52
2. GitLab Environment Support for Multi-Environment Pipelines	53
Strategy for Resolving Configuration URLs per Environment	53
3. Inter-Repository Authorisation	53

Authorisation Requirements	53
Mechanisms for Overriding Default Values in spec:inputs:	53
Example: Overriding Variables for Protected Environments	54
4. Public Container Registry and Secrets Availability	55
Secrets Mapping Table	55
Example: podman push in CI/CD and Local Development	56
5. Parallel Container Image Builds	56
Job Graph and Failure-Aware Dependency Chain	56
.gitlab-ci.yml Sketch	56
6. DevOps Strategy Synthesis	58
Operative Procedures	58
Comparison of Alternatives	59
References	59
Additional Notes	60
Security Considerations	60
Performance Optimizations	60
Local Development Parity	60
Multi-Repository Workflow	60
Python Project Automation with uv	60
Example: Full .gitlab-ci.yml for Downstream Project	61
Key Takeaways	63
Next Steps	64
A:a	65
A:a - .gitlab-ci.yml Example	65
Appendix a - initial .gitlab-ci.yml GitLab pipeline	65

TOC

1. Q:1 - GitLab CI CD Multi Repo Configuration Strategy
 - see: [GitLab CI/CD multi-repo setup \(Claude\)](#)
 - see: [GitLab CI/CD Multi-Repo Strategy \(Gemini\)](#)
 - see: [Multi-Repository GitLab CI/CD Strategy \(ChatGPT\)](#)
 - see: [GitLab CI CD Multi Repo Configuration Strategy \(DeepSeek\)](#)
 - see: [Multi-repository GitLab CI/CD strategy \(Perplexity\)](#)
 - see: [GitLab CI/CD Multi-Repository Strategy for Python Projects \(Vibe\)](#)
2. A:a - `.gitlab-ci.yml` Example (Appendix-a)

Q:1

Q:1 - GitLab CI CD Multi Repo Configuration Strategy

^

Role

You are an expert in GitLab CI/CD pipelines for multi-repository environments, specialising in:

- targeting testing and deployment across multiple environments (development, testing, staging, production)
- authorising CI/CD pipeline jobs across a multi-repository setup (public upstream repository, private downstream fork, and public GitHub mirror)
- Python project automation using `uv`, including pre-release validation with `pyright`, `ruff`, and `pytest`, and post-release container image build and delivery via Podman to a public container registry
- DevOps best practices for private Git-based, multi-environment configuration management and versioning

Terminology

To avoid ambiguity, use these terms consistently throughout your response:

- *upstream* — the canonical public repository
- *downstream* — the private fork, extended with unmerged features
- *mirror* — the public GitHub copy of the upstream repository

Context

Two GitLab projects share the same Python codebase, managed with `uv`:

- *Public upstream project*: `ub-dems-public/ds-lab/dve-simple-py`, branch `main`
- *Private downstream project*: `ub-dems/vs-base/dve-simple-py`, branches `main` and `development`
- *Public mirror project* (GitHub): `ub-dems/dve-simple-py`, branch `main`

Requirements to address:

- inter-repository authorisation so that pipelines in the downstream project can trigger merge requests and tag jobs on the upstream project, and push the codebase to the mirror
- public container registry credentials, available as CI/CD pipeline variables

- environment-dependent configuration data (API access tokens, database credentials), available consistently across CI/CD pipelines, local development, and application runtime

Before answering, reason through each environment (development, testing, staging, production) and each repository (upstream, downstream, mirror) as a distinct combination, so that environment-specific and repository-specific nuances are not collapsed into a single generic answer.

Objective

1. Configuration management best practices

- constraint: all configuration data must reside in private GitLab repositories
- constraint: avoid duplication between GitLab CI/CD variables and local development/run-time configuration
- *deliverable*: a pros/cons comparison of `git submodule` versus a parallel checkout of a configuration repository (accessed via relative path, no symbolic links), with a final recommendation

2. GitLab environment support for multi-environment pipelines

- pipeline targets: development, testing, staging, production
- *deliverable*: a strategy, with a supporting table (environment → configuration source URL → resolution mechanism), for resolving the correct configuration URL per environment

3. Inter-repository authorisation (GitLab-to-GitLab and GitLab-to-GitHub)

- constraint: authorisation is required only for CI/CD pipelines, not for local development
- *deliverable*: an explanation of how default values declared in a `spec:inputs:` header can be overridden at pipeline trigger time, including which override mechanisms apply to protected branches/environments

4. Public container registry and secrets availability

- constraint: `podman push` must work identically in GitLab pipelines and local development
- constraint: the same applies to API access tokens and database credentials
- *deliverable*: a table mapping each secret type to its storage location and retrieval mechanism, per context (pipeline vs. local)

5. Parallel container image builds

- constraint: each container image is built as an independent parallel job
- constraint: the job graph must implement a failure-aware dependency chain
- *deliverable*: a description (or `.gitlab-ci.yml` sketch) of the job graph and its failure-handling behaviour

6. DevOps strategy synthesis

- propose concrete operative procedures covering objectives 1–5
- discuss critical points and compare the pros and cons of the alternatives considered
- provide references to relevant online materials (articles, tutorials, discussions)

Output Format

Reply in clear, lint-valid GitLab Flavored Markdown (GLFM):

- place codeblock delimiters (“”) at the start of the line; avoid nested codeblocks
- use *underscore markup* for emphasis
- prefer nested headings over asterisk-based text markup
- use dashes only for unordered lists, with correct indentation
- insert a blank line after every heading, list, and codeblock

Disregard the following non-standard formatting markup if encountered elsewhere:

- `<details><summary>` HTML blocks
- `{=latex}` codeblocks
- `[!tip]` / `[!note]` blockquotes
- triple-curly-brace code-folding tags
- internal links (e.g. `[^]`)

At the end of the response, list references to online documentation as Markdown footnotes, prefixed with `rf-` to avoid clashing with other references in the document, and link them inline to the relevant answer text.

Add any additional relevant information not explicitly requested, under an *Additional Notes* section.

A:1 (Claude)

^ GitLab CI/CD multi-repo setup

===(Sonnet-5 [medium])=====

For managing CI/CD across multiple GitLab repositories, there are several complementary strategies depending on how much sharing and coordination you need. Here's the landscape:

1. Shared/Centralized CI Templates

Keep a dedicated repo (e.g. ci-templates or devops/ci-common) with reusable .yaml files, then pull them into each project:

```
include:
  - project: 'devops/ci-templates'
    ref: main
    file: '/templates/node-build.yaml'
```

Good for standardizing lint/test/build/deploy jobs across many services without duplicating logic. Pin ref to a tag/branch for stability, or main if you want changes to propagate immediately (riskier).

2. CI/CD Components (GitLab 16.9+)

The modern evolution of shared templates — versioned, discoverable via the Component Catalog, with declared inputs:

```
include:
  - component: gitlab.com/my-group/ci-components/docker-build@1.2.0
    inputs:
      image_name: my-service
```

This is now GitLab's recommended approach over raw template includes for reusable, parameterized CI logic.

3. Multi-Project Pipelines (trigger jobs)

When repos are related and need to trigger each other's pipelines (e.g., a library repo triggering downstream consumers' builds):

```
deploy_downstream:
  stage: deploy
```

```
trigger:
  project: my-group/downstream-service
  branch: main
  strategy: depend
```

strategy: depend makes the upstream pipeline wait on the downstream result — useful for coordinated releases.

4. Parent-Child Pipelines

Different from multi-project — this splits *one repo's* pipeline into dynamically generated child pipelines. Less about multi-repo, but often combined with the above when a monorepo triggers per-service child pipelines.

5. Group-Level Shared Configuration

- **Group CI/CD variables:** define secrets/config once at the group level instead of per-project
- **Compliance pipelines / pipeline execution policies** (Ultimate tier): enforce mandatory jobs (security scans, etc.) across all projects in a group, injected regardless of each repo's own `.gitlab-ci.yml`
- **Instance/group runners:** shared runner fleet tagged appropriately so repos don't each need their own

6. Repo Topology Considerations

- **Many small repos (microservices):** lean on CI/CD Components + trigger jobs for cross-repo coordination
- **Few large repos:** rules: changes: to scope jobs to changed paths, reducing need for cross-repo triggers
- **Mixed:** some teams keep a "platform" repo owning shared component versions, and downstream repos bump the version like a dependency

A common production setup: a ci-components repo owning versioned components → each service repo includes specific versions → group variables for shared secrets → trigger jobs only where deploy ordering actually matters (e.g., infra before app).

What's your situation — are you dealing with many microservice repos needing consistent CI, or a few repos that need to coordinate deploys with each other? That'll help narrow down which pattern fits best.

===(Sonnet-5 [high])===

Environment × Repository Combinations

Before the individual objectives, here is the reasoning grid this answer is built on — it is referenced throughout instead of repeating the same environment/repo caveats in every section.

Environment	Repository	Ref / trigger	Notes
<i>development</i>	downstream	development branch, every push	fastest feedback, no image public
<i>testing</i>	downstream	main branch (post-merge from development)	full pyright / ruff / pytest gat
<i>staging</i>	upstream	main branch (post-merge of downstream's MR)	release-candidate image, not yet
<i>production</i>	upstream	v* tags	final image build, push to the reg
—	mirror	main (GitHub)	passive publication target only; i

Keeping this grid explicit matters because "environment" and "repository" are two independent axes here — *testing* happens on the *downstream* repo, but *staging* and *production* only exist on *upstream*. Collapsing them into one generic development → ... → production ladder would misrepresent where each pipeline actually runs.

1. Configuration Management Strategy

The two constraints — all configuration in private GitLab repositories, and no duplication between CI/CD variables and local/runtime configuration — point at a specific design: environment-dependent secrets (API tokens, database credentials) should not live as GitLab CI/CD variables at all. They should live as files in a dedicated private configuration repository, and both the pipeline and the local/runtime environment should resolve them the *same way* (a checkout), not through two independently-maintained stores.

That leaves the actual question of objective 1: how does that private configuration repository get attached to `dve-simple-py`?

`git submodule`

- Pros
 - Version-pinned by design — the parent repo's tree records an exact commit SHA of the config repo, so `git checkout <tag>` on the app repo deterministically reproduces the matching config state.
 - Familiar Git-native tooling; no bespoke bootstrap script required for the checkout step itself.
 - Works uniformly whether cloned by a runner or a developer, as long as credentials for the submodule URL are present.
- Cons
 - Requires the cloning credential (deploy token or SSH key) to be available to `git submodule update --init` in *every* context, including the runner's shallow clone strategy, which needs `GIT_SUBMODULE_STRATEGY: recursive` and often `GIT_SUBMODULE_FORCE_HTTPS` tweaks to avoid SSH-vs-HTTPS URL rewriting inside CI.
 - Detached-HEAD submodule state is a recurring source of confusion for contributors, and bumping the pointer is an extra commit that's easy to forget.
 - `.gitmodules` merge conflicts are common across the development → main → upstream main promotion path described in the environment grid above.

- Ties the configuration ref to the *app repo's* commit graph, which is awkward when the same app commit needs to run against different environments' configuration (dev pipeline and staging pipeline checking out the same app SHA but different config refs).

Parallel checkout (sibling directory, relative path)

- Pros
 - No Git metadata coupling — a plain `git clone --branch <ref> <config-repo-url> ../dve-simple-py-config` step, identical in a CI job's `before_script` and in a developer's local bootstrap.
 - The ref to check out is a simple string (environment name, tag, branch) rather than a SHA baked into the app repo's tree, so the *same* app commit can be run against development, testing, staging, or production configuration without touching the app repo at all — which matches the requirement that configuration be environment-dependent rather than app-version-dependent.
 - Avoids the "no symbolic links" constraint naturally, since the app's configuration loader just resolves a relative path (`../dve-simple-py-config/environments/<env>/`) rather than needing a symlinked `config/` directory inside the app tree.
 - Easier to reason about for the three-repository topology: downstream, upstream, and local checkouts can each point their sibling clone at a different ref of the *same* config repo without any of that logic living inside `.gitmodules`.
- Cons
 - No automatic version pin — the "current" config ref has to be tracked deliberately (a `CONFIG_REF` value, not a Git-recorded pointer), so an explicit convention is required or the binding between an app release and its expected config state can drift.
 - Requires an explicit clone step in CI (can't rely on `GIT_SUBMODULE_STRATEGY`), and requires the same bootstrap logic to be documented/scripted for local development so the sibling-directory assumption is honoured consistently.

Recommendation

Use the parallel checkout, with the ref made explicit rather than implicit: keep a single `CONFIG_REF` (a branch or tag name per environment, e.g. `environments/staging`) as the one thing that is intentionally duplicated — as a pointer, not as secret material — between a small `Makefile`/bootstrap script and a CI/CD variable. The bootstrap script is the single implementation used by both the pipeline (invoked as a `.pre-stage` job) and local development (invoked by the developer), so the actual DB/API credential values are never copied anywhere except inside the config repository itself. This satisfies both constraints: everything sensitive stays in a private repo, and there is exactly one code path that resolves it, eliminating the classic "CI/CD variable says one thing, `.env` file says another" drift. `git submodule` is reasonable if you want cryptographic version-pinning of config-to-app-release, but for a case explicitly organised around *environments* rather than *releases*, the parallel checkout is the better fit.

2. Multi-Environment Configuration Resolution

Given the grid in the introduction, "which configuration source" is really a function of two things: which repository the pipeline is running in, and which ref triggered it. The cleanest way to make that explicit — rather than inferring it from `CI_COMMIT_REF_NAME` string matching scattered across the pipeline — is to declare it as a typed pipeline input using `spec:inputs`¹, with `options:` constraining it per repository (see objective 3 for the override mechanics).

```
spec:
  inputs:
    environment:
      type: string
      default: development
      options: [development, testing, staging, production]
---
resolve-config:
  stage: .pre
  script:
    - export CONFIG_REF="environments/${[ inputs.environment ]}"
    - git clone --branch "$CONFIG_REF" "$CONFIG_REPO_URL" ../dve-simple-py-config
  artifacts:
    paths:
      - ../dve-simple-py-config
```

Environment	Repository	Configuration source (ref)
development	downstream, development	environments/development branch of the config repo
testing	downstream, main	environments/testing branch
staging	upstream, main	environments/staging tag (pinned, not a moving branch)
production	upstream, v* tag	environments/production tag, gated by a protected environment a

Two distinct mechanisms are layered here deliberately: the `spec:inputs` value decides *which configuration files get checked out*, while GitLab's native `environment:` keyword on the actual deploy/publish job is what enables environment-scoped CI/CD variables² and protected-environment approvals for anything that *does* need to live as a GitLab-native variable (see objective 4). They answer two different questions — "what config does this job read" versus "is this job allowed to act on this environment" — and conflating them tends to produce pipelines where a config mix-up and an authorisation failure look identical in the logs.

¹GitLab Docs, "[CI/CD Inputs](#)"

²GitLab Docs, "[Environments](#)" ([environment-scoped variables](#))

3. Inter-Repository Authorisation

Downstream → Upstream (merge requests and tags)

Because this needs to reach GitLab's REST API (create an MR, create a tag/release) rather than only trigger a pipeline, a project access token on *upstream* is the right primitive — `CI_JOB_TOKEN` is deliberately narrow in what API surface it can reach³ (pipeline triggers, package/container registry, artifacts — not general MR/tag creation), whereas a project access token carries the `api` scope needed for those calls.

- Create a bot-owned project access token on *upstream*, scoped to the minimum role that can open MRs and push tags (Developer is usually enough; tag protection rules may require Maintainer).
- Store it only in *downstream*'s CI/CD variables, marked *Protect variable* and *Masked*, so it's only materialised on downstream's protected main branch pipelines — never on development, and never available to a developer's shell.
- Use it from a dedicated job (`git remote add upstream https://<bot>:$UPSTREAM_API_TOKEN@...` plus the Merge Requests / Tags API) gated by rules: `if: $CI_COMMIT_BRANCH == "main"`.

If any part of the downstream→upstream interaction also needs to *trigger a pipeline* on upstream directly (rather than relying on the tag push itself to start upstream's own pipeline), add downstream to upstream's CI/CD job token allowlist⁴ as a second, narrower channel reserved for that specific use — keep the two credentials distinct rather than reusing the API token for both purposes, since the allowlist grants a different (and more constrained) blast radius than a project access token.

Downstream → Mirror (GitHub)

Two viable approaches, worth comparing rather than picking one blindly:

- Native push mirroring, configured in *upstream*'s project settings (Settings → Repository → Mirroring)⁵, with a GitHub PAT stored in GitLab's mirror configuration (not exposed as a CI/CD variable or in job logs at all). Simple, and it's GitLab-managed rather than custom pipeline code — but it runs on GitLab's own schedule/webhook, so it isn't choreographed against the rest of the release pipeline (e.g. "only mirror after the production image has successfully published").
- A pipeline job (in downstream, per your stated requirement) that pushes explicitly, using a fine-grained GitHub PAT (`contents:write` on the mirror repo only) stored as a protected+masked CI/CD variable. This gives you pipeline-level control — e.g. `needs:` it on the upstream tag/release job succeeding — at the cost of one more secret to manage and one more script to maintain.

Given the stated requirement that mirror pushes are part of the downstream pipeline's authorised actions, the second approach fits better if mirror timing must be coordinated with the release; the first is simpler if "GitHub always mirrors upstream main" is sufficient. Either way, this credential should live only where the mirroring job runs, never in local development.

³GitLab, "CI/CD job token" reference

⁴GitLab, "CI/CD job token" reference

⁵GitLab Docs, "Repository mirroring"

spec:inputs overrides at trigger time

spec:inputs values declared in a `.gitlab-ci.yml` header are fixed once the pipeline is created and can't drift mid-run the way a CI/CD variable can⁶ — which is precisely why they're the better mechanism here for something like environment, where you want a hard contract rather than a value that could be reassigned by a later job.

Override paths, and which are relevant across the three repos:

- `trigger:inputs` — used on the multi-project trigger job when downstream's promotion job triggers upstream's pipeline, e.g. `trigger: { project: ub-dems-public/ds-lab/dve-simple-py, inputs: { environment: staging } }`.
- `include:inputs` — for composing a single pipeline's own configuration from included templates/components; not relevant to cross-project triggering itself, but relevant if upstream and downstream both consume a shared CI template repo.
- Manual override via the "Run pipeline" UI or the pipelines API — used by a release manager kicking off a production run by hand.

The protected-branch/protected-environment interaction is layered, not a single switch:

- Ref protection (`main`, `v*` on upstream being protected branches/tags) determines *who is even allowed to trigger a pipeline* there in the first place — only someone with push/merge rights on the protected ref can supply any input at all.
- `options:` on the input definition constrains *which values* are legal regardless of who's triggering — e.g. upstream's `spec:inputs` can restrict `environment` to `[staging, production]`, so even an authorised maintainer can't accidentally point a protected-branch pipeline at development configuration.
- Protected environments (Operate → Environments → Protect) are a separate, later gate on the actual deployment job — even a pipeline that correctly resolved `environment: production` can still require named approvers before the job executes.

4. Container Registry and Secrets Availability

The constraint that `podman push` and every other secret must behave identically in pipelines and local development is best served by *not* trying to literally copy the same secret value between the two contexts, but by giving each context its own credential of equivalent scope, resolved through the same mechanism.

Secret	Storage location
Registry push credential	Group-level protected + masked CI/CD variables (<code>CI_REGISTRY_PUSH_USER</code> / ...)
Config-repo access credential	Deploy token (<code>read_repository</code> scope) on the config repo, stored as protected
Per-environment API tokens	Files inside the private config repository (<code>environments/<env>/api.env</code>)
Per-environment DB credentials	Same config repository, same layout

The one point worth flagging explicitly: since the registry is *public*, none of this may ever be baked

⁶GitLab Docs, "[CI/CD Inputs](#)"

into an image layer as a build-time ARG/ENV — image contents are visible to anyone who pulls it. Runtime secrets have to be injected at deploy time (a mounted config checkout, an orchestrator secret, or environment variables set outside the Containerfile), never inlined into the image build.

5. Parallel Container Image Builds With Failure-Aware Dependencies

`needs:` gives exactly the semantics wanted here: jobs listed under `needs:` don't have to share a stage boundary with jobs that don't depend on them, so independent image builds run in parallel, and a downstream job only starts once *all* of its listed dependencies have succeeded⁷. Critically, one build job failing does not cancel its siblings — they run to completion, so a broken image build doesn't hide the status of the others — it only blocks the job(s) that explicitly `needs:` the failed one.

```
stages:
  - build
  - publish

.build-template:
  stage: build
  image: quay.io/podman/stable
  script:
    - podman build -t "${CI_REGISTRY_IMAGE}/${IMAGE_NAME}:${CI_COMMIT_SHORT_SHA}" -f "containers/$
    - podman save -o "${IMAGE_NAME}.tar" "${CI_REGISTRY_IMAGE}/${IMAGE_NAME}:${CI_COMMIT_SHORT_SHA}
  artifacts:
    paths:
      - "${IMAGE_NAME}.tar"
    expire_in: 1h
  retry: 1

build-images:
  extends: .build-template
  parallel:
    matrix:
      - IMAGE_NAME: [dve-simple-cli, dve-simple-api, dve-simple-worker]

publish-images:
  stage: publish
  needs:
    - build-images
  script:
    - podman login "${CI_REGISTRY}" -u "${CI_REGISTRY_PUSH_USER}" -p "${CI_REGISTRY_PUSH_TOKEN}"
    - for f in *.tar; do podman load -i "$f"; done
    - podman push "${CI_REGISTRY_IMAGE}/dve-simple-cli:${CI_COMMIT_SHORT_SHA}"
    - podman push "${CI_REGISTRY_IMAGE}/dve-simple-api:${CI_COMMIT_SHORT_SHA}"
    - podman push "${CI_REGISTRY_IMAGE}/dve-simple-worker:${CI_COMMIT_SHORT_SHA}"
```

⁷GitLab Docs, "[CI/CD YAML syntax reference](#)" (`needs`, `parallel:matrix`)

```
rules:
  - if: $CI_COMMIT_TAG
```

`needs: [build-images]` referencing a `parallel:matrix` job by its base name depends on *all* of its generated instances⁸, so `publish-images` correctly waits on the full matrix rather than just the first instance to finish. If `dve-simple-worker`'s build fails, `dve-simple-cli` and `dve-simple-api` still finish and report their own status individually, but `publish-images` is skipped entirely — nothing partially publishes. `retry: 1` on the build template absorbs transient runner/network flakiness without masking a genuine build failure, which is what actually blocks `publish-images`.

6. Synthesis

Putting objectives 1–5 together, the pattern that emerges is: minimise how many places a secret's *value* lives, and be deliberate about which access-control layer (protected branch, `spec:inputs` options, protected environment, job token allowlist, project access token) is doing which job, since they're easy to conflate and each covers a different failure mode.

- Configuration: one private config repository, resolved by parallel checkout in both CI and local development, rather than `git submodule` — chosen because environment selection needed to be independent of the app repo's commit graph.
- Cross-repo authorisation: a project access token for anything hitting the GitLab API (MRs, tags) on *upstream*, kept out of local development entirely; `CI_JOB_TOKEN` allowlisting reserved specifically for pipeline-to-pipeline triggering, not API calls; a separate, narrowly-scoped GitHub PAT for the mirror push, weighed against GitLab's built-in push mirroring depending on whether mirror timing needs to be pipeline-choreographed.
- Secrets: registry push credentials as CI/CD variables (pipeline-only, with developers using their own personal credentials rather than a shared one); runtime secrets (API tokens, DB credentials) as config-repo files, not CI/CD variables, because the running application — not just the pipeline — needs them, and GitLab CI/CD variables aren't visible outside a job's runtime.
- Environments: a typed `spec:inputs` value drives which configuration is checked out; GitLab's native `environment:` keyword and protected environments drive who's allowed to actually deploy, layered on top of protected-branch ref restrictions.
- Parallel builds: `parallel:matrix` plus `needs:` for isolation-without-blocking — each image's failure is visible on its own, and only the aggregate publish step is gated.

The main trade-off worth re-stating: this design favours explicitness (a bootstrap script, a documented `CONFIG_REF` convention, several distinct credentials) over the apparent simplicity of “just put everything in CI/CD variables and a submodule.” That costs a small amount of extra tooling up front, but it's what actually satisfies the stated constraints — private-repo-only configuration, no CI/local duplication, and pipeline-only authorisation — rather than approximating them.

⁸GitLab Docs, “CI/CD YAML syntax reference” ([needs, parallel:matrix](#))

Additional Notes

- If the shared bootstrap/config-resolution logic ends up duplicated between upstream and downstream `.gitlab-ci.yml` files, it's a good candidate for a versioned CI/CD Component published to a component catalog, consumed with `include: component:` and its own `spec:inputs`, rather than a copy-pasted `include: project: template` — that gives both repos a single upgrade path for the resolution logic itself.
- `uv` fits naturally into the parallel-build job template as the tool used inside each `Containerfile`'s build stage; keeping `pyright/ruff/pytest` as an earlier, non-matrixed stage means a lint/type-/test failure blocks all three image builds before any `podman build` even starts, which is usually cheaper than letting all three builds run and fail independently.
- Because upstream is public, anonymous read access already exists for the mirror and for anyone inspecting the codebase — the authorisation work in objective 3 is entirely about *write* operations (MRs, tags, pushes), not about protecting read access to the code itself.

References

- see⁹: [GitLab Docs, "CI/CD Inputs"](#)
- see¹⁰: [GitLab Docs, "CI/CD YAML syntax reference" \(needs, parallel:matrix\)](#)
- see¹¹: [GitLab, "CI/CD job token" reference](#)
- see¹²: [GitLab, "CI Protected Variables"](#)
- see¹³: [GitLab Docs, "Environments" \(environment-scoped variables\)](#)
- see¹⁴: [GitLab Docs, "Repository mirroring"](#)

⁹GitLab Docs, ["CI/CD Inputs"](#)

¹⁰GitLab Docs, ["CI/CD YAML syntax reference" \(needs, parallel:matrix\)](#)

¹¹GitLab, ["CI/CD job token" reference](#)

¹²GitLab, ["CI Protected Variables"](#)

¹³GitLab Docs, ["Environments" \(environment-scoped variables\)](#)

¹⁴GitLab Docs, ["Repository mirroring"](#)

A:1 (Gemini)

^ *GitLab CI/CD Multi-Repo Strategy*

Pre-analysis: Repository and Environment Matrix

Before defining the concrete architecture, it is necessary to map out how the environments and repositories interact, as they have fundamentally different lifecycle responsibilities.

The Upstream Repository (Public)

The *upstream* repository serves as the public face of the project. It handles stable releases and accepts community contributions.

- development: N/A (development happens downstream)
- testing: Public CI runs (e.g., `uv run pytest`, `ruff check`) on merge requests from external contributors
- staging: N/A
- production: Triggered on tag creation to publish public releases (e.g., PyPI packages or public container images)

The Downstream Repository (Private)

The *downstream* repository is where the core team works. It contains proprietary extensions, sensitive configuration integrations, and the deployment pipeline logic.

- development: Continuous integration on the development branch; tests against development infrastructure
- testing: Run on feature branches or merge requests targeting development/main
- staging: Pre-release validation on the main branch before merging upstream or deploying
- production: Private production deployments, triggered manually or upon syncing upstream tags

The Mirror Repository (Public GitHub)

The *mirror* repository is a passive reflection of the *upstream*. No internal CI/CD pipelines run here; it acts purely as a distribution and visibility channel for GitHub users.

Objective 1: Configuration Management Best Practices

Storing environment-dependent configuration (API tokens, database credentials) securely while maintaining parity between local development and CI/CD requires separating configuration from the codebase.

Git Submodule vs. Parallel Checkout

- *Git Submodule*
 - Pros: Ties a specific commit of the codebase to a specific commit of the configuration, ensuring exact reproducibility. Native to Git.
 - Cons: Highly intrusive. Adds complexity to merge requests. CI/CD pipelines require special `GIT_SUBMODULE_STRATEGY` configuration. Developers frequently encounter detached HEAD states. It is also risky for a public *upstream* repository, as the submodule pointer would expose the existence and structure of the private configuration repository.
- *Parallel Checkout*
 - Pros: Completely decouples the public codebase from private configuration. Configuration is cloned into a sibling directory during CI or local setup. The public *upstream* remains completely unaware of the configuration repository.
 - Cons: Requires an explicit setup script or task (e.g., via `uv run` or a `Makefile`) to fetch the correct configuration branch. Version synchronization is manual or managed via CI/CD variables.

Recommendation

A *parallel checkout* is the superior approach for this architecture. Because the *upstream* repository is public, embedding a submodule pointing to a private downstream configuration repository will break for public contributors. By using a parallel checkout, developers and CI runners explicitly clone the configuration repository into a sibling folder (e.g., `../dve-simple-py-config`) only when authorized, keeping the main repository clean.

Objective 2: GitLab Environment Support

GitLab Environments allow pipelines to dynamically fetch the correct configuration based on the target deployment stage.

Configuration Resolution Strategy

The *downstream* pipeline determines the environment context based on Git rules (e.g., `development` branch maps to the `development` environment). A single GitLab CI/CD variable, `CONFIG_REPO_URL`, is overridden at the environment level in GitLab's project settings.

Environment Resolution Table

- development
- URL: `git@gitlab.internal/ub-dems/config/dev.git`
- Mechanism: Sibling checkout triggered by branch pipeline on development branch
- testing
- URL: `git@gitlab.internal/ub-dems/config/test.git`
- Mechanism: Sibling checkout triggered by Merge Request pipelines in *downstream*
- staging
- URL: `git@gitlab.internal/ub-dems/config/staging.git`
- Mechanism: Sibling checkout triggered by commits to main in *downstream*
- production
- URL: `git@gitlab.internal/ub-dems/config/prod.git`
- Mechanism: Sibling checkout triggered by tags in *downstream*

Objective 3: Inter-repository Authorisation

Authorisation must be seamless across the multi-repository setup for CI/CD, without encumbering local development.

GitLab to GitLab Authorisation

To allow the *downstream* pipeline to trigger jobs on the *upstream* pipeline, utilise GitLab CI/CD Job Tokens (`CI_JOB_TOKEN`) in conjunction with multi-project pipeline triggers. The *upstream* repository must whitelist the *downstream* repository in its token access settings ¹⁵.

GitLab to GitHub Authorisation (Mirroring)

To push codebase updates to the *mirror*, generate a GitHub Personal Access Token (PAT) or deploy key with repo scope, and store it as a masked, protected variable in the *upstream* GitLab project.

Pipeline Trigger Overrides and `spec:inputs`

When using CI/CD components, `spec:inputs` define default values (e.g., target image registry). While `include:inputs` are resolved at configuration parse time, you can design your components to accept inputs that map to pipeline variables.

When triggering a pipeline via the API or a multi-project trigger, you can pass variables that override the defaults. If the target branch or environment is marked as *protected* in GitLab, the variables

¹⁵[GitLab Job Token documentation](#)

passed in the trigger will only take effect if the user or token initiating the trigger has permission to execute pipelines on that protected entity. Otherwise, the trigger will fail ¹⁶.

Objective 4: Public Container Registry and Secrets Availability

To ensure podman push and python application execution work identically locally and in CI/CD, secrets must be injected through the environment (env).

Secrets Storage and Retrieval Table

- Registry Credentials
- Location (CI): GitLab Masked Variables (REGISTRY_USER, REGISTRY_PASS)
- Location (Local): Local HashiCorp Vault or podman login local keystore
- Retrieval: Expose variables to the podman CLI execution script
- API Access Tokens
- Location (CI): GitLab Protected Variables bound to Environments
- Location (Local): .env file loaded via python-dotenv or uv run --env-file
- Retrieval: Read via os.environ in Python
- Database Credentials
- Location (CI): CI/CD Vault Integration or Environment Variables
- Location (Local): .env file
- Retrieval: Read via os.environ in Python

By standardising on environment variables as the single source of truth at runtime, the application code remains completely agnostic to whether it is running locally or in GitLab CI.

Objective 5: Parallel Container Image Builds

Building multi-architecture or multi-component images should happen in parallel to save time, using a failure-aware DAG (Directed Acyclic Graph).

Job Graph Behaviour

You can define independent build jobs using parallel:matrix. If one build variant fails (e.g., the arm64 build), it should not necessarily stop the deployment of the amd64 build in non-production environments, but it must halt production releases.

¹⁶[GitLab CI/CD Components and Inputs](#)

Job Graph Sketch

```

build_images:
  stage: build
  parallel:
    matrix:
      - ARCH: [amd64, arm64]
  script:
    - podman build --arch $ARCH -t registry.example.com/dve-simple-py:${CI_COMMIT_SHA}-${ARCH} .
    - podman push registry.example.com/dve-simple-py:${CI_COMMIT_SHA}-${ARCH}

manifest_creation:
  stage: release
  needs:
    - job: build_images
      artifacts: false
  script:
    - podman manifest create registry.example.com/dve-simple-py:latest
    - podman manifest add registry.example.com/dve-simple-py:latest registry.example.com/dve-simple-py:latest
    - podman manifest push registry.example.com/dve-simple-py:latest

```

By leveraging `needs:`, the `manifest_creation` job will only execute if all parallel instances of `build_images` succeed, creating a failure-aware chain.

Objective 6: DevOps Strategy Synthesis

Operative Procedures

1. Code is pushed to development on the *downstream* repo.
2. The CI pipeline uses a script to check out the parallel configuration repository based on the environment.
3. Code is validated using `uv run ruff check` and `uv run pytest`.
4. Feature branches are merged to main *downstream*.
5. When ready for release, a trigger job on *downstream* uses a project access token to push the changes to *upstream* via a Merge Request.
6. Once merged *upstream*, a tag is created. The *upstream* tag pipeline builds the public images via Podman and pushes the code to the GitHub *mirror*.

Critical Points

Managing the sync between *upstream* and *downstream* is the most fragile part of this architecture. If *upstream* receives community contributions, *downstream* must frequently rebase to avoid merge conflicts. Using a strict "rebase-only" strategy for the downstream repository mitigates this risk.

Pros and Cons Summary

- Parallel Config Checkout: Highly flexible, keeps upstream clean (Pro). Requires extra local setup scripts (Con).
- Environment Variables for Secrets: Ultimate portability between CI and local (Pro). `.env` files can accidentally be committed locally if `.gitignore` is misconfigured (Con).

Additional Notes

When using `uv` in CI/CD, ensure you utilise its caching capabilities. Set the `UV_CACHE_DIR` environment variable to a directory within the project (e.g., `.uv-cache`) and configure GitLab CI to cache this directory across pipeline runs. This will significantly reduce pipeline duration by avoiding redundant downloads of Python dependencies and tools like `ruff` and `pyright`.

References

- see¹⁷: [GitLab Job Token documentation](#)
- see¹⁸: [GitLab CI/CD Components and Inputs](#)

¹⁷[GitLab Job Token documentation](#)

¹⁸[GitLab CI/CD Components and Inputs](#)

A:1 (ChatGPT)

^ *Multi-Repository GitLab CI/CD Strategy for a Python uv Project*

Executive Summary

The architecture described consists of three repositories with different responsibilities:

Repository	Visibility	Purpose	Priority
<i>upstream</i>	Public GitLab	Canonical source, releases, tags	main
<i>downstream</i>	Private GitLab	Internal development, environment-specific integration, unreleased features	development
<i>mirror</i>	Public GitHub	Public distribution mirror	main

The four deployment environments have distinct characteristics:

Environment	Repository of Record	Typical Trigger	Secrets Required
<i>development</i>	downstream	developer push	development
<i>testing</i>	downstream	merge request / scheduled	testing
<i>staging</i>	upstream (pre-release) or downstream release branch	tag candidate	staging
<i>production</i>	upstream	protected tag	production

The most maintainable solution is:

- keep *all configuration* inside private Git repositories
- separate *code* from *configuration*
- use a dedicated private configuration repository checked out in parallel (not as a Git submodule)
- use GitLab protected environments together with protected CI/CD variables
- use pipeline inputs only for selecting the target environment—not for carrying secrets
- let every pipeline retrieve the correct configuration automatically
- authenticate GitLab-to-GitLab using *Project Access Tokens* or *CI_JOB_TOKEN* where supported
- authenticate GitLab-to-GitHub using a GitHub Fine-Grained Personal Access Token or GitHub App token
- build container images completely independently and use DAG dependencies only for publication

1. Configuration Management

Goals

The configuration mechanism should satisfy:

- single source of truth
 - identical behaviour in:
 - local development
 - GitLab CI
 - runtime
 - no duplication of configuration values
 - secrets never committed into application repositories
 - reproducible historical configurations
-

Option A — Git Submodule

Repository layout:

```
dve-simple-py/  
  .gitmodules  
  config/
```

Advantages

- exact configuration revision recorded in every commit
- reproducible historical builds
- normal Git workflow

Disadvantages

- additional Git commands

```
git submodule update --init --recursive
```

- developers frequently forget to update submodules
 - merge conflicts involving submodule pointers
 - CI becomes more complicated
 - nested repositories confuse IDE indexing
 - awkward when multiple projects should share one configuration repository
-

Option B — Parallel Checkout (*Recommended*)

Workspace:

workspace/

dve-simple-py/

dve-simple-config/

Configuration is accessed using relative paths:

../dve-simple-config

No symbolic links.

Advantages

- configuration repository has independent lifecycle
- reusable across many projects
- developers can switch configuration branches independently
- CI simply clones another repository
- IDEs index normally
- easier automation

Disadvantages

- workspace convention required
- project itself no longer records exact configuration commit

The latter is usually solved by:

- pinning configuration branch names
- recording configuration commit hash inside build metadata

Recommendation

For this architecture, a *parallel checkout* is the preferable approach because:

- configuration evolves independently from application code
 - several projects can reuse the same configuration repository
 - CI remains straightforward
 - local development and runtime use exactly the same layout
-

2. Multi-Environment Configuration Resolution

A dedicated repository may have a layout such as

config/

development/

testing/

staging/

production/

or

config/

env-development.toml

env-testing.toml

env-staging.toml

env-production.toml

Resolution Strategy

Instead of hardcoding repository URLs inside scripts, define a single environment selector.

Example:

TARGET_ENV=development

Pipeline determines:

1. target environment
 2. configuration repository URL
 3. configuration branch
 4. configuration directory
-

Resolution Table

Environment	Configuration Repository URL	Resolution Mechanism
development	private GitLab configuration repository	selected by pipeline input or branch name
testing	same repository	selected by pipeline input
staging	same repository	selected by protected environment
production	same repository	selected only from protected tags

Example:

variables:

```
CONFIG_REPOSITORY: git@gitlab.example.org:config/dve-simple-config.git
```

Pipeline:

```
git clone "$CONFIG_REPOSITORY"
```

```
cd dve-simple-config
```

```
git checkout "$TARGET_ENV"
```

or

```
cp environments/$TARGET_ENV/config.toml ...
```

This keeps one repository while separating environment data.

3. Inter-Repository Authorisation

GitLab → GitLab

Operations required:

- create merge requests
- create tags
- trigger pipelines
- clone private repositories

Recommended credentials:

Operation	Recommended Mechanism
clone private repository	CI_JOB_TOKEN where supported
trigger downstream pipeline	Pipeline Trigger Token or Project Access Token
create MR	Project Access Token
create tag	Project Access Token

Project Access Tokens provide narrower scope than Personal Access Tokens.¹⁹

GitLab → GitHub

Mirror updates require authentication.

Recommended:

- GitHub Fine-Grained PAT
- or GitHub App

Store:

GITHUB_TOKEN

as protected GitLab variable.

Pipeline:

```
git push https://oauth2:${GITHUB_TOKEN}@github.com/...
```

spec:inputs Default Override

Example:

```
spec:
  inputs:
    target_environment:
      default: development
```

Pipeline:

```
include:
- project: pipeline-library
  file: deploy.yml
  inputs:
    target_environment: staging
```

Default values may be overridden by:

- include:inputs
- parent pipelines
- pipeline trigger API
- manual "Run pipeline"
- scheduled pipelines

Protected branches and protected environments restrict *who may supply override values*, but the

¹⁹GitLab Docs — [Project Access Tokens](#)

override mechanism itself remains the same.²⁰

For example:

Trigger Source	Can Override Inputs	Notes
Push pipeline	No	Uses defaults
Manual pipeline	Yes	User permissions apply
Trigger token	Yes	Token permissions apply
Parent pipeline	Yes	Explicit inputs: mapping
Pipeline API	Yes	API caller permissions apply
Protected tag pipeline	Yes	Only authorized users may create the protected tag

Pipeline inputs should control deployment behaviour (such as selecting development, testing, staging, or production) rather than carrying secrets.

4. Secrets Strategy

Principles

Secrets should exist only in:

- configuration repository
- GitLab protected variables

The application reads secrets through a common loader.

Storage Matrix

Secret	Pipeline Storage	Local Storage
Container registry credentials	GitLab protected variables	private configuration repository or
API tokens	encrypted/private configuration repository	same repository
Database credentials	encrypted/private configuration repository	same repository
GitHub token	GitLab protected variable	not required
GitLab Project Access Token	GitLab protected variable	not required

The application should expose one configuration interface, for example:

```
settings = load_configuration()
```

which internally resolves values from:

²⁰GitLab Docs — [CI/CD Inputs \(spec:inputs\)](#)

1. environment variables
2. configuration repository
3. defaults

This makes runtime and CI behave identically.

5. Parallel Container Image Builds

Assume:

- base
- documentation
- runtime
- development

Each image is independent.

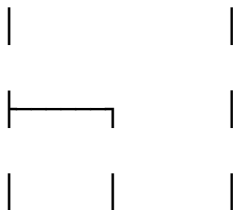
Example DAG:

validate

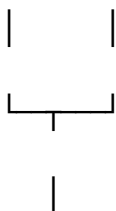


build-base

build-runtime



build-dev build-docs



publish-images

|

release

Example:

validate:

stage: validate

build-runtime:

stage: build

needs:

- validate

build-base:

stage: build

needs:

- validate

build-docs:

stage: build

needs:

- validate

build-dev:

stage: build

needs:

- validate

publish:

stage: publish

needs:

- build-runtime

- build-base

- build-docs

- build-dev

Failure Behaviour

- if validation fails, nothing builds
- if one image fails:
 - remaining independent builds continue
 - publication does not start
- release jobs never execute until every image succeeds

This minimises wasted CI time while preserving release integrity.

6. Overall DevOps Strategy

Repository Responsibilities

downstream

- active development
- feature branches
- integration testing
- development deployments

upstream

- canonical releases
- tags
- production deployments

mirror

- receive pushes only after upstream release succeeds
-

Recommended Operational Workflow

1. Developers work only in the *downstream* repository.
2. Every pipeline validates with:
 - ruff
 - pyright
 - pytest
3. Successful changes are proposed to the *upstream* through an automated merge request.
4. Release tags are created only in the *upstream* repository.
5. The release pipeline:
 - checks out the matching configuration
 - builds images in parallel
 - pushes images using podman
 - deploys to the selected environment
 - updates the GitHub *mirror*
6. Production deployments are initiated only from protected tags targeting the production environment.

Critical Considerations

Parallel Checkout

-Pros*

- simpler automation
- reusable configuration
- no submodule complexity
- easier IDE support

-Cons*

- requires a consistent workspace layout
- configuration revision should be recorded explicitly for reproducibility

Protected Environments

-Pros*

- centralized authorization
- clear audit trail
- environment-specific secrets
- minimizes accidental production deployments

-Cons*

- additional administrative setup
- requires disciplined branch and tag protection

Project Access Tokens

-Pros*

- scoped permissions
- rotation independent of user accounts
- suitable for automation

-Cons*

- lifecycle management is required
 - permissions should be kept as narrow as practical
-

Additional Notes

- Keep the application repository completely free of environment-specific data; the only environment selector should be the deployment target.

- Prefer GitLab *File* CI/CD variables only for small generated artifacts (for example, signing keys or certificates). Larger structured configuration is generally easier to version, review, and audit in the private configuration repository.
- Record both the application commit SHA and the configuration repository commit SHA in build metadata (for example, OCI image labels). This significantly improves traceability and reproducibility during incident analysis.
- Consider using GitLab *Components* and reusable pipeline templates to centralize common validation, release, and deployment logic across repositories, reducing duplication while allowing environment-specific inputs.²¹
- For Podman authentication, standardize on `podman login` using environment variables in CI and the same variables (or a generated `auth.json`) in local development so that `podman push` behaves identically in both contexts.
- If the configuration repository contains sensitive material, consider encrypting only the secret values (for example, using Mozilla SOPS with an external key management system) while keeping the repository structure and non-sensitive configuration in plain text. This preserves version control benefits without exposing secrets in Git history.

References

- see²²: GitLab Docs — [Project Access Tokens](#)
- see²³: GitLab Docs — [CI/CD Inputs \(*spec:inputs*\)](#)
- see²⁴: GitLab Docs — [CI/CD Components](#)
- see²⁵: GitLab Docs — [Environments and Protected Environments](#)
- see²⁶: GitLab Docs — [CI/CD Variables](#)
- see²⁷: GitLab Docs — [CI_JOB_TOKEN](#)
- see²⁸: GitLab Docs — [Directed Acyclic Graph Pipelines \(*needs*\)](#)
- see²⁹: Podman Documentation — [podman login](#)
- see³⁰: Astral - [uv Documentation](#)
- see³¹: GitHub Docs — [Managing Fine-grained Personal Access Tokens](#)
- see³²: Mozilla - [SOPS Documentation](#)

²¹GitLab Docs — [CI/CD Components](#)

²²GitLab Docs — [Project Access Tokens](#)

²³GitLab Docs — [CI/CD Inputs \(*spec:inputs*\)](#)

²⁴GitLab Docs — [CI/CD Components](#)

²⁵GitLab Docs — [Environments and Protected Environments](#)

²⁶GitLab Docs — [CI/CD Variables](#)

²⁷GitLab Docs — [CI_JOB_TOKEN](#)

²⁸GitLab Docs — [Directed Acyclic Graph Pipelines \(*needs*\)](#)

²⁹Podman Documentation — [podman login](#)

³⁰Astral - [uv Documentation](#)

³¹GitHub Docs — [Managing Fine-grained Personal Access Tokens](#)

³²Mozilla - [SOPS Documentation](#)

A:1 (DeepSeek)

^ *GitLab CI CD Multi Repo Configuration Strategy*

TODO:(a1-deepseek) ...

Preliminary Analysis: Environment × Repository Combinations

Before designing pipelines and configuration strategies, the distinct roles of each environment and repository must be mapped out. The table below clarifies where different concerns intersect.

Environment	Upstream (ub-dems-public/ds-lab/dve-simple-py)	Downstream (ub-dems-public/ds-lab/dve-simple-py)
development	Not directly used; receives merge requests from downstream.	Primary development environment.
testing	Pipeline triggered by downstream to run integration tests on candidate code.	Triggers upstream development.
staging	Runs staging deployment after manual approval, using tag or release branch.	Triggers upstream development.
production	Holds release tags; pipeline builds and pushes final images.	Initiates release process.

This matrix underlies all subsequent decisions: configuration must be accessible in downstream CI (all environments), upstream CI (testing, staging, production), and locally during development. Inter-repository authorisation is needed from downstream → upstream (triggers) and from upstream → mirror (push). Credentials and configuration must flow seamlessly across these boundaries.

1. Configuration Management Best Practices

Constraint

- All configuration data lives in private GitLab repositories.
- Duplication between GitLab CI/CD variables and local development/runtime configuration must be avoided.

The Two Approaches

Both approaches keep the configuration in a dedicated private GitLab repository (`config-private`). The difference lies in how the main repository references it.

A) Git Submodule

The main repository contains a `.gitmodules` entry pointing to `config-private`. Developers and CI check it out with `git submodule update --init --recursive`.

B) Parallel Checkout (Sibling Directory)

The CI pipeline clones config-private into a path relative to the main checkout (e.g., ../config-private). Locally, developers are instructed to clone the config repo manually into a sibling directory. No symlinks; the application reads ../config-private/<env>/ using a relative path.

Comparison

Criterion	Git Submodule
Consistency	Pins a specific commit; repo state is reproducible.
Setup Complexity	Requires --recurse-submodules on clone; submodule pointer updates are an extra step.
Developer Experience	Often leads to detached HEAD confusion and accidental submodule drift.
CI Integration	Submodule checkout is a single CI step.
Duplication Risk	Low – config lives only in the submodule; variables are never duplicated in GitLab CI vars.
Flexibility	Changing config requires committing to config repo and updating the submodule pointer.

Recommendation

Parallel checkout with environment-specific branches in the configuration repository.
It decouples configuration lifecycle from the application code, simplifies environment mapping (branch = environment name), and avoids the friction of submodules. The small cost of an explicit clone instruction for developers is justified by the clarity it provides across all environments and repositories.

2. Multi-Environment Configuration Resolution

The private configuration repository (config-private) holds one branch per environment: development, testing, staging, production. Each branch contains the environment-specific files (e.g., .env, credentials encrypted with SOPS, Kubernetes manifests). Pipelines resolve the correct branch using a mapping from the predefined CI variable \$CI_ENVIRONMENT_NAME.

Environment	Configuration Source URL (example)	Resolution Mechanism
development	git@gitlab.com:private/config-private.git (branch development)	CI job sets CONFIG_BRANCH=development
testing	git@gitlab.com:private/config-private.git (branch testing)	Same mapping; triggered by test pipeline
staging	git@gitlab.com:private/config-private.git (branch staging)	Upstream staging pipeline
production	git@gitlab.com:private/config-private.git (branch production)	Release pipeline checks out production

The pipeline includes a common “config checkout” job that runs before any environment-dependent step:

```
checkout-config:
  stage: .pre
```

```

script:
  - git clone --branch "$CONFIG_BRANCH" --single-branch \
    "git@gitlab.com:private/config-private.git" ../config-private
artifacts:
  paths:
    - ../config-private/

```

All subsequent jobs obtain configuration by reading `../config-private/`. The application uses the same relative path at runtime.

3. Inter-Repository Authorisation

a) GitLab-to-GitLab: Downstream Triggers Upstream

The downstream repository needs to trigger pipelines in the upstream project (e.g., for merge request validation and tagging). Authorization uses a **GitLab project access token** with `trigger_pipeline` scope, stored as a CI/CD variable in the downstream project (`UPSTREAM_TRIGGER_TOKEN`).

Trigger Job with Input Override

Upstream CI defines a pipeline with `spec:inputs:` to accept parameters:

```

# upstream .gitlab-ci.yml
spec:
  inputs:
    environment:
      default: testing
    ref:
      default: main
---
triggered-pipeline:
  script:
    - echo "Running tests for $[[ inputs.environment ]] on ref $[[ inputs.ref ]]"

```

The downstream triggers it using the `trigger` keyword, passing variables that override the defaults:

```

# downstream .gitlab-ci.yml
trigger-upstream-tests:
  stage: test
  trigger:
    project: ub-dems-public/ds-lab/dve-simple-py
    branch: main
    strategy: depend
  variables:
    INPUT_environment: testing          # overrides default
    INPUT_ref: $CI_COMMIT_SHA

```


When a `spec:inputs:` key is named `environment`, GitLab automatically maps a variable `INPUT_environment` to the input. Default values declared in `spec:inputs:` are only used when no corresponding variable is passed.

Protected branch/environment restrictions:

If the upstream pipeline runs on a protected branch (e.g., `main`) or a protected environment, the token used for the trigger must belong to a user with at least **Maintainer** role on the upstream project. A project access token can be granted that role. The downstream project itself may be private; only the token privileges matter.

b) GitLab-to-GitHub: Mirror Push

The upstream pipeline pushes the codebase to the public GitHub mirror. Authorization relies on a **GitHub deploy key** (read-write) stored as a GitLab CI variable (`GITHUB_DEPLOY_KEY`). A dedicated job adds the key and pushes:

```
mirror-to-github:
  stage: mirror
  script:
    - mkdir -p ~/.ssh
    - echo "$GITHUB_DEPLOY_KEY" > ~/.ssh/id_ed25519
    - chmod 600 ~/.ssh/id_ed25519
    - ssh-keyscan github.com >> ~/.ssh/known_hosts
    - git remote add mirror git@github.com:ub-dems/dve-simple-py.git
    - git push mirror $CI_COMMIT_SHA:refs/heads/main
  only:
    - tags          # or main branch
```

This authorisation is active only in CI pipelines, satisfying the constraint.

4. Public Container Registry and Secrets Availability

All secrets (container registry credentials, API tokens, database credentials) are stored **encrypted** inside the private configuration repository, e.g., using `sops` with an age key. The same decryption key is made available to CI pipelines via a masked GitLab variable, and to developers through their local environment. This ensures identical workflows for `podman push` and secret consumption in every context.

Secret Type	Storage Location	Retrieval Method
Container registry credentials	<code>../config-private/<env>/registry.enc.yaml</code> (sops-encrypted)	CI variable <code>SOPS_REGISTRY_CREDENTIALS</code>
API access tokens	<code>../config-private/<env>/api-tokens.enc.env</code>	Same decryption key as registry credentials
Database credentials	<code>../config-private/<env>/db.enc.yaml</code>	Decrypted as <code>DB_CREDENTIALS</code>

In a pipeline job, the flow is:

build-and-push:

script:

```
- apt-get update && apt-get install -y sops
- sops --decrypt ../config-private/$CI_ENVIRONMENT_NAME/registry.enc.yaml > registry-creds.sh
- source registry-creds.sh
- podman login -u "$PODMAN_USER" -p "$PODMAN_PASS" quay.io
- podman build -t quay.io/ub-dems/dve-simple-py:$CI_COMMIT_TAG .
- podman push quay.io/ub-dems/dve-simple-py:$CI_COMMIT_TAG
```

Locally, a developer follows the same steps, ensuring podman push behaves identically.

5. Parallel Container Image Builds with Failure-Aware Dependency Chain

The pipeline builds multiple images (e.g., app, worker, sidecar) as independent parallel jobs. A subsequent publish job collects artifacts only from successful builds, avoiding a single failure from blocking the entire release.

Job Graph Description

- **Stage build:** Contains a matrix job (or individual parallel jobs) for each image variant. Each job is set `allow_failure: true` so that a failure in one does not cancel the others.
- **Stage publish:** A single job that needs all build jobs with `optional: true` (GitLab ≥ 16.3). Its script inspects the downloaded artifacts and pushes only those images whose artifacts exist.

Sketch of .gitlab-ci.yml

stages:

```
- build
- publish
```

.image-build-template: &image-build

stage: build

image: quay.io/podman/stable

allow_failure: true

script:

```
- sops --decrypt ../config-private/$CI_ENVIRONMENT_NAME/registry.enc.yaml > creds.sh
- source creds.sh
- podman login -u "$PODMAN_USER" -p "$PODMAN_PASS" quay.io
- |
  IMAGE_NAME=$(echo "$CI_JOB_NAME" | cut -d: -f2)
  podman build -t quay.io/ub-dems/dve-simple-py/${IMAGE_NAME}:$CI_COMMIT_SHA -f Dockerfile.$
  podman push quay.io/ub-dems/dve-simple-py/${IMAGE_NAME}:$CI_COMMIT_SHA
```

artifacts:

paths:

```

    - image-built.${IMAGE_NAME}    # empty marker file

build-image:app:
  <<: *image-build
  variables:
    IMAGE_NAME: app

build-image:worker:
  <<: *image-build
  variables:
    IMAGE_NAME: worker

build-image:sidecar:
  <<: *image-build
  variables:
    IMAGE_NAME: sidecar

publish-release:
  stage: publish
  image: quay.io/podman/stable
  needs:
    - job: build-image:app
      artifacts: true
      optional: true
    - job: build-image:worker
      artifacts: true
      optional: true
    - job: build-image:sidecar
      artifacts: true
      optional: true
  script:
    - sops --decrypt ../config-private/${CI_ENVIRONMENT_NAME}/registry.enc.yaml > creds.sh
    - source creds.sh
    - podman login -u "$PODMAN_USER" -p "$PODMAN_PASS" quay.io
    - |
      for img in app worker sidecar; do
        if [ -f image-built.${img} ]; then
          podman tag quay.io/ub-dems/dve-simple-py/${img}:${CI_COMMIT_SHA} \
            quay.io/ub-dems/dve-simple-py/${img}:${CI_COMMIT_TAG}
          podman push quay.io/ub-dems/dve-simple-py/${img}:${CI_COMMIT_TAG}
        else
          echo "Image ${img} was not built; skipping release tag."
        fi
      done

```

Failure handling behaviour:

- If any build job fails, the marker file is absent. The publish-release job still runs because of `optional: true`, but that image variant is skipped.
 - The pipeline's overall status can be set to "failed" if a critical image is missing. The publish job can implement a check and exit non-zero when a required image is absent (e.g., app).
 - This design avoids a "single point of failure" blocking all deliverables while still allowing enforcement of quality gates.
-

6. DevOps Strategy Synthesis**Concrete Operative Procedures****1. Configuration Repository Setup**

- Create `config-private` with branches `development`, `testing`, `staging`, `production`.
- Store all non-secret config as plain files; encrypt secrets with `sops` (age key).
- Distribute the decryption key: as a protected CI variable `SOPS_AGE_KEY` in the downstream project, and locally to developers via secure channel (password manager).

2. Local Development

- Clone `dve-simple-py` and `config-private` as sibling directories.
- Export `SOPS_AGE_KEY` in shell profile.
- Run `sops decrypt ../config-private/development/...` before launching the app.
- Execute `podman` commands using the same decrypted credentials.

3. CI/CD Pipeline – Downstream (development / testing)

- `.pre` stage clones `config-private` branch `$CI_ENVIRONMENT_NAME`.
- Decrypt secrets, run `pyright`, `ruff`, `pytest`.
- For testing environment, trigger upstream pipeline with `INPUT_environment=testing` using a project access token.

4. CI/CD Pipeline – Upstream (staging / production)

- Triggered from downstream on merge request or tag.
- Clones `config-private` with the appropriate environment branch.
- Runs integration tests, builds container images in parallel.
- On tags: tags release images, pushes to public registry, pushes code to GitHub mirror.

5. Release Flow

- Developer merges into downstream `development` → CI tests.
- When ready, a merge to `main` (downstream) triggers a staging pipeline upstream.
- After manual approval, a tag is created downstream, triggering the production pipeline upstream that builds, pushes images, and updates the mirror.

Critical Points and Trade-offs

- **Security of the age key:** The single SOPS_AGE_KEY unlocks all secrets. It must be stored in a protected GitLab variable (masked, restricted to protected branches) and never committed. Revoking it requires rotating all secrets.
- **Parallel checkout vs. submodule:** While parallel checkout is simpler, it relies on CI and developers maintaining the sibling directory convention. A CI job `clone-config` must be present in every pipeline. Missing this step leads to hard-to-diagnose failures; documentation and pipeline templates mitigate this.
- **Failure-aware builds:** Using `optional: true` prevents one broken image from halting the entire release. However, it demands careful scripting to decide which images are mandatory and fail the pipeline accordingly.
- **Authorization tokens:** Project access tokens have expiration dates; rotation must be automated or monitored. GitHub deploy keys are permanent but can be revoked easily.
- **Environment mapping:** The convention `branch name = environment name` is intuitive, but branches like `production` need strict protection (no direct pushes). Only the CI job should check out that branch.

Comparison of Alternatives

- **Configuration duplication:** Storing secrets directly in GitLab CI/CD variables would require syncing with local development (e.g., `.env` files in the repo), violating the no-duplication constraint and risking drift.
- **Git-crypt instead of SOPS:** Git-crypt works transparently but does not support structured file-level encryption and can be harder to rotate keys. SOPS with age keys is more flexible and widely adopted in CI contexts.
- **Triggering upstream via API instead of trigger keyword:** The `trigger` keyword with `strategy: depend` allows the downstream pipeline to reflect upstream status, which is more declarative and requires less scripting than API calls.

References

- see³³: GitLab documentation, [“Spec:inputs – Pipeline inputs for triggered pipelines.”](#)
- see³⁴: GitLab documentation, [“Multi-project pipelines.”](#)
- see³⁵: GitLab documentation, [“Job artifacts and needs with optional.”](#)
- see³⁶: GitLab documentation, [“Project access tokens.”](#)
- see³⁷: Mozilla SOPS – [“Secrets OPERATION.”](#)
- see³⁸: Podman documentation, [“podman login and podman push.”](#)

³³GitLab documentation, [“Spec:inputs – Pipeline inputs for triggered pipelines.”](#)

³⁴GitLab documentation, [“Multi-project pipelines.”](#)

³⁵GitLab documentation, [“Job artifacts and needs with optional.”](#)

³⁶GitLab documentation, [“Project access tokens.”](#)

³⁷Mozilla SOPS – [“Secrets OPERATION.”](#)

³⁸Podman documentation, [“podman login and podman push.”](#)

Additional Notes

- The pre stage that clones the configuration repository should also verify the checkout integrity by checking a known file or commit signature.
- For local development, a Makefile target or a wrapper script (`./scripts/decrypt-config.sh`) can standardize the decryption step and avoid manual errors.
- Consider using rules to limit the mirror push and production image builds strictly to protected tags, reducing the risk of accidental pushes.
- The parallel build strategy can be extended with `parallel:matrix` to dynamically generate image variants, reducing duplication.

A:1 (Perplexity)

^ Multi-repository GitLab CI/CD strategy

Below is a concrete strategy for your *upstream* / *downstream* / *mirror* setup, with environment-specific handling for development, testing, staging, and production, and with a recommendation that keeps configuration private, avoids duplication, and makes CI/CD authorization explicit. GitLab inputs are the cleanest way to parameterize environment selection, while protected and environment-scoped CI/CD variables should hold secrets and runtime credentials. [docs.gitlab](#)

1) Configuration management

The best fit is a *parallel checkout* of a private configuration repository, mounted by relative path in both local development and CI, rather than a `git submodule`. GitLab variables support environment scopes and protection, but values stored only in CI/CD settings are not a good single source of truth for local runtime; GitLab also recommends keeping sensitive values out of `.gitlab-ci.yml` and using protected/masked variables or file variables in settings instead. [docs.gitlab](#)

Submodule vs parallel checkout

Approach	Pros
<code>git submodule</code>	Strong coupling to a pinned commit; reproducible; simple if all consumers use the same commit
Parallel checkout of a private config repo	Same file tree can be consumed by CI jobs, local dev, and runtime; can be updated without CI

Recommendation: use a parallel checkout, with the config repository cloned into a fixed sibling directory and resolved by relative path from the application repo. This gives one configuration tree that can be used identically by pipelines, local development, and the running app, while still keeping all config private in GitLab as required. [docs.gitlab](#)

2) Environment resolution

Treat development, testing, staging, and production as separate resolution targets, each with its own configuration source URL. Use a single input such as `target_environment`, declared in `spec:inputs`, with allowed values limited to those four environments; GitLab inputs are validated at pipeline creation, and defaults should be provided so automatic pipelines never fail because a value was omitted. [docs.gitlab](#)

Environment	Configuration source URL	Resolution mechanism
development	Private config repo path for development config	Pipeline input default or manual override; conf
testing	Private config repo path for testing config	Same input contract; testing jobs map to a test
staging	Private config repo path for staging config	Same contract; staging deployment job uses a s
production	Private config repo path for production config	Same contract; production jobs use protected,

A practical pattern is to have one config repo containing `environments/development`, `environments/testing`, `environments/staging`, and `environments/production`, with the application or deployment code resolving `CONFIG_URL` or `CONFIG_PATH` from the selected environment. Because environment-scoped variables are assigned only at job execution time, do not rely on them for `spec:inputs` interpolation; instead, use inputs to choose the environment and variables only for secrets and runtime data. [docs.gitlab](#)

3) Cross-project authorization

For CI/CD only, use a dedicated token strategy per direction: the *downstream* project should authenticate to the *upstream* project with a pipeline trigger token or a job-token allowlist, and it should authenticate to the *mirror* with a GitHub deploy key or fine-scoped GitHub token stored as masked/protected CI/CD variables in GitLab. GitLab's variable precedence means pipeline variables and trigger variables override project and group variables, so you should keep overrides explicit and constrained. [docs.gitlab](#)

spec:inputs: override behavior

A `spec:inputs:` header defines default values that apply unless a trigger-time value is supplied. Inputs can be set when running a pipeline manually, through the pipeline triggers API, through downstream pipeline forwarding, or via `trigger:inputs`; for included configs, use `include:inputs`. If a pipeline uses protected branches or protected tags, protected variables remain available only on those protected refs, and protected environments can further restrict who may deploy to development, testing, staging, and production. [docs.gitlab](#)

Important nuance: inputs and variables are different layers. Inputs are fixed during pipeline creation and are the right mechanism for selecting environment or repo target, while CI/CD variables are the right mechanism for secrets, credentials, and runtime materialization. If you need an override for a protected branch or tag, prefer a protected pipeline variable or a protected trigger variable; if you need a controlled parameter, prefer an input with a default and a restricted option list. [docs.gitlab](#)

4) Secrets and registry access

Use the same secret names everywhere, but change how they are delivered: in GitLab pipelines, use protected/masked project or group variables, environment-scoped where appropriate; locally, use a private config checkout plus a local env file or credential helper that the application reads with the same variable names. GitLab notes that file-type variables are useful when a tool expects a file path, and that secrets should be masked and, where appropriate, protected. [docs.gitlab](#)

Secret type	Pipeline storage
Public container registry credentials	Protected masked CI/CD variables in GitLab project/group settings
API access tokens	Protected masked variable, optionally environment-scoped
Database credentials	Protected masked variable, optionally as file-type variable if a client expects

To keep podman push identical in CI and locally, standardize on one auth bootstrap script that reads REGISTRY_USER, REGISTRY_PASSWORD, and REGISTRY_URL from the environment, with the CI job getting them from GitLab variables and local development getting them from the private config checkout or shell exports. That way the command sequence is the same in both contexts, and only the secret source differs. [docs.gitlab](#)

5) Parallel image builds

Model each image as an independent build job, then gate deployment jobs through explicit needs: edges so the graph is parallel where possible but failure-aware where required. GitLab needs lets jobs start earlier, while artifact-dependency behavior must be handled carefully when jobs are parallelized; GitLab's docs and related issue discussion show that artifact handling and parallel builds need explicit design, especially when downstream jobs depend on the results of multiple build jobs. [docs.gitlab](#)

Job graph sketch

stages:

- validate
- build
- publish
- deploy

validate:

stage: validate

script:

- uv sync --locked
- pyright
- ruff check .
- pytest

build_api_image:

stage: build

needs: ["validate"]

script:

- podman build -f Containerfile.api -t "\$IMAGE_API:\$CI_COMMIT_SHA" .

artifacts:

paths:

- image-api.ref

```
build_worker_image:
  stage: build
  needs: ["validate"]
  script:
    - podman build -f Containerfile.worker -t "$IMAGE_WORKER:$CI_COMMIT_SHA" .
  artifacts:
    paths:
      - image-worker.ref

publish_api_image:
  stage: publish
  needs:
    - job: build_api_image
      artifacts: true
  script:
    - podman login "$REGISTRY_URL" -u "$REGISTRY_USER" -p "$REGISTRY_PASSWORD"
    - podman push "$IMAGE_API:$CI_COMMIT_SHA"

publish_worker_image:
  stage: publish
  needs:
    - job: build_worker_image
      artifacts: true
  script:
    - podman login "$REGISTRY_URL" -u "$REGISTRY_USER" -p "$REGISTRY_PASSWORD"
    - podman push "$IMAGE_WORKER:$CI_COMMIT_SHA"

deploy_staging:
  stage: deploy
  needs:
    - job: publish_api_image
    - job: publish_worker_image
  script:
    - ./deploy.sh staging
```

Failure handling should be strict for validation, build, and publish jobs, and deployment should only start when all required publish jobs succeed. If one image is optional for a given environment, put that image behind a separate rule set or make the downstream deploy job depend only on the required publishes; do *not* use `allow_failure` for build jobs unless you explicitly want partial releases. [docs.gitlab](#)

6) Operative procedures

A workable operating model is:

- Keep the config repo private and mirrored nowhere else.
- Use the *downstream* project as the orchestration point for CI/CD.
- Use `spec:inputs` to select repository target and environment, with defaults that make scheduled, push, and merge-request pipelines safe to create automatically. [docs.gitlab](#)
- Use protected variables for registry and secret values, and environment-scoped variables for per-environment runtime materialization. [docs.gitlab](#)
- Use protected environments for deployment jobs targeting staging and production, so only approved roles can deploy there. [docs.gitlab](#)
- Use one credential bootstrap script for both CI and local podman push, so behavior stays identical.
- Use separate parallel build jobs for each image, then publish, then deploy only after all required publishes succeed. [docs.gitlab](#)

Repository-specific nuances

- *Upstream*: accept authorized CI-triggered merge request or tag jobs from *downstream* only through narrowly scoped tokens and allowlists; avoid broad access.
- *Downstream*: owns the working CI/CD graph, environment selection, secret usage, and release orchestration.
- *Mirror*: receive only post-release pushes or sync jobs; keep GitHub credentials out of the codebase and inside protected GitLab variables.
- *development*: relaxed promotion, but still use private config and secret separation.
- *testing*: validate release candidates against a dedicated config slice before staging.
- *staging*: use protected variables and protected environment approval.
- *production*: same as staging, but with stricter environment protection and the smallest possible variable surface. [docs.gitlab](#)

Critical points

The main trade-off is between convenience and control. `git submodule` is familiar, but a parallel checkout is better aligned with your requirement that the same configuration must serve CI, local development, and runtime without duplication. The other important risk is overusing pipeline variables; GitLab explicitly notes that pipeline variables can override other values and recommends inputs for typed, validated pipeline parameters instead. [docs.gitlab](#)

Another critical point is secret scope. Masking helps with log leakage, but it is not a complete defense against malicious `.gitlab-ci.yml` changes, so treat protected branches, protected environments, and code review of CI changes as part of the security boundary. For cross-repository authorization, keep tokens per destination and per purpose, rather than using one broad credential for *upstream* and *mirror* access. [docs.gitlab](#)

References

- see[^ ap-1] [GitLab CI/CD inputs](#).
- see[^ ap-2] [GitLab CI/CD variables](#).
- see[^ ap-3] [Protected environments](#).

- see[^ ap-4] [Trigger pipelines with the API](#).
- see[^ ap-5] [GitLab pipelines overview](#).
- see[^ ap-6] [needs keyword reference](#).
- see[^ ap-7] [Job artifacts](#).
- see[^ ap-8] [Cross-project pipelines](#).

[^ ap-1] [GitLab CI/CD inputs](#). [^ ap-2] [GitLab CI/CD variables](#). [^ ap-3] [Protected environments](#). [^ ap-4] [Trigger pipelines with the API](#). [^ ap-5] [GitLab pipelines overview](#). [^ ap-6] [needs keyword reference](#). [^ ap-7] [Job artifacts](#). [^ ap-8] [Cross-project pipelines](#).

A:1 (Vibe)

^ *GitLab CI/CD Multi-Repository Strategy for Python Projects*

1. Configuration Management Best Practices

Comparison: `git submodule` vs. Parallel Checkout of a Configuration Repository

Criteria	<code>git submodule</code>
Versioning	Explicit: submodule references a specific commit, ensuring reproducibility.
Privacy	Supports private repositories, but submodule URLs are exposed in <code>.gitmodules</code> .
Ease of Use	Complex: requires <code>git submodule update --init --recursive</code> ; can confuse contributors.
Duplication	Avoids duplication: submodule is a reference, not a copy.
CI/CD Integration	Works well in CI/CD but requires shallow clones and careful handling of submodule checkout.
Local Development	Requires contributors to manage submodules locally.
Maintenance Overhead	Higher: updating submodules requires commits in the main repo.
Security	Submodule URLs are visible in <code>.gitmodules</code> ; may leak private repo paths if not carefully managed.

Final Recommendation

Use a parallel checkout of a private configuration repository (accessed via relative path, no symbolic links).

Rationale:

- **Privacy:** The configuration repository remains private, and its path is not exposed in the main repository.
- **Simplicity:** No need for contributors or CI/CD pipelines to handle submodules, reducing complexity and potential for errors.
- **Flexibility:** The configuration repository can be updated independently of the main repository, and different environments (e.g., development, staging) can reference different branches or tags of the config repo.
- **CI/CD Friendliness:** Easier to clone the config repo alongside the main repo in CI/CD jobs, avoiding the need for nested Git operations.

Implementation:

- Store the configuration repository in a private GitLab project (e.g., `ub-dems/vs-base/dve-simple-py-config`).

- In CI/CD pipelines and local development, clone the config repo alongside the main repo using a relative path (e.g., ../dve-simple-py-config).
- Use CI/CD variables to dynamically set the config repo branch/tag based on the target environment (e.g., main for production, development for development).

2. GitLab Environment Support for Multi-Environment Pipelines

Strategy for Resolving Configuration URLs per Environment

Environment	Configuration Source URL	Reso
development	git@gitlab.com:ub-dems/vs-base/dve-simple-py-config.git (branch: development)	CI/CD
testing	git@gitlab.com:ub-dems/vs-base/dve-simple-py-config.git (branch: testing)	CI/CD
staging	git@gitlab.com:ub-dems/vs-base/dve-simple-py-config.git (branch: staging)	CI/CD
production	git@gitlab.com:ub-dems/vs-base/dve-simple-py-config.git (tag: v1.0.0)	CI/CD

Key Points:

- The **configuration repository** is private and shared across all environments.
- The **branch/tag** of the config repo is determined by the target environment, using CI/CD variables.
- **Local development** mirrors the CI/CD approach: clone the config repo alongside the main repo and check out the appropriate branch/tag.
- **No duplication:** The same config repo is used for all environments; only the reference (branch/tag) changes.

3. Inter-Repository Authorisation

Authorisation Requirements

- **CI/CD Pipelines Only:** Authorisation is required only for CI/CD pipelines (not local development).
- **Targets:**
 - Downstream → Upstream: Trigger merge requests and tag jobs.
 - Downstream → Mirror: Push code to the GitHub mirror.

Mechanisms for Overriding Default Values in spec:inputs:

GitLab CI/CD supports overriding default values declared in spec:inputs: (or variables) at pipeline trigger time using:

1. Pipeline Variables (UI/API):

- Override via the GitLab UI when manually triggering a pipeline.
- Override via the GitLab API when triggering a pipeline programmatically.
- Example: Override `UPSTREAM_REPO_URL` or `GITHUB_MIRROR_URL` for a specific run.

2. Protected Branches/Environments:

- For protected branches (e.g., `main`, `production`), use **protected CI/CD variables** (masked and available only to protected branches/environments).
- Override mechanisms:
 - **Project-level variables:** Set in Settings > CI/CD > Variables and marked as protected.
 - **Group-level variables:** Inherited by all projects in the group, with protection options.
 - **Pipeline schedules:** Define variables for scheduled pipelines.
 - **Merge request pipelines:** Use rules or `only/except` to conditionally set variables for MR pipelines.

3. Trigger Tokens:

- Use **trigger tokens** to start pipelines with predefined variables (e.g., for downstream → upstream triggers).
- Example:

```
# In downstream .gitlab-ci.yml
trigger_upstream:
  stage: deploy
  trigger:
    project: ub-dems-public/ds-lab/dve-simple-py
    token: $UPSTREAM_TRIGGER_TOKEN
    strategy: depend
    variables:
      SOURCE_BRANCH: $CI_COMMIT_REF_NAME
```

4. GitHub Mirror Push:

- Use a **personal access token (PAT)** or **deploy key** for the GitHub mirror, stored as a protected CI/CD variable (e.g., `GITHUB_MIRROR_TOKEN`).
- Override the remote URL in the pipeline:

```
push_to_mirror:
  script:
    - git remote add mirror https://oauth2:$GITHUB_MIRROR_TOKEN@github.com/ub-dems/dve-
    - git push mirror $CI_COMMIT_REF_NAME
```

Example: Overriding Variables for Protected Environments

```
# .gitlab-ci.yml (downstream)
stages:
```

```

- test
- deploy

variables:
  UPSTREAM_REPO_URL: "git@gitlab.com:ub-dems-public/ds-lab/dve-simple-py.git"
  GITHUB_MIRROR_URL: "https://github.com/ub-dems/dve-simple-py.git"

# Override for production
production_deploy:
  stage: deploy
  rules:
    - if: $CI_COMMIT_REF_NAME == "main"
  variables:
    UPSTREAM_REPO_URL: "git@gitlab.com:ub-dems-public/ds-lab/dve-simple-py.git" # Can be overri
    GITHUB_MIRROR_URL: "https://oauth2:$GITHUB_MIRROR_TOKEN@github.com/ub-dems/dve-simple-py.git
  script:
    - echo "Deploying to production using $UPSTREAM_REPO_URL"

```

Override at Trigger Time:

- When manually triggering a pipeline for production, override `UPSTREAM_REPO_URL` or `GITHUB_MIRROR_URL` in the GitLab UI.
- For API-triggered pipelines, pass variables in the request:
`curl --request POST --form "variables[UPSTREAM_REPO_URL]=new_url" https://gitlab.com/api/v4`

4. Public Container Registry and Secrets Availability

Secrets Mapping Table

Secret Type	Pipeline Storage	Pipeline Retrieval
Container Registry Credentials	Protected CI/CD variable (REGISTRY_USER, REGISTRY_TOKEN)	echo \$REGISTRY
API Access Tokens	Protected CI/CD variable (API_TOKEN)	echo \$API_TOKE
Database Credentials	Protected CI/CD variable (DB_USER, DB_PASS)	echo \$DB_PASS

Key Constraints:

- **podman push:** Must work identically in pipelines and locally. Use the same environment variables (e.g., `REGISTRY_USER`, `REGISTRY_TOKEN`) in both contexts.
- **Local Development:** Use a `.env` file (excluded from Git via `.gitignore`) to store secrets locally. Load them using `python-dotenv` or similar tools.
- **CI/CD Pipelines:** Use **protected, masked CI/CD variables** for all secrets. These are only available to jobs running on protected branches/environments.

Example: podman push in CI/CD and Local Development**CI/CD Pipeline:**

```
build_and_push:
  stage: deploy
  script:
    - echo $REGISTRY_TOKEN | podman login -u $REGISTRY_USER --password-stdin $CI_REGISTRY
    - podman build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_TAG .
    - podman push $CI_REGISTRY_IMAGE:$CI_COMMIT_TAG
  rules:
    - if: $CI_COMMIT_TAG # Only run for tags
```

Local Development:

```
# .env file
REGISTRY_USER=myuser
REGISTRY_TOKEN=mytoken

# Terminal
source .env
podman login -u $REGISTRY_USER --password-stdin $REGISTRY_TOKEN $CI_REGISTRY
podman build -t myimage:latest .
podman push myimage:latest
```

5. Parallel Container Image Builds

Job Graph and Failure-Aware Dependency Chain**Requirements:**

- Each container image is built as an **independent parallel job**.
 - The job graph must implement a **failure-aware dependency chain** (i.e., if a build fails, dependent jobs should not run).
-

.gitlab-ci.yml Sketch

```
stages:
  - test
  - build
  - deploy

# Shared variables
variables:
```

```

IMAGE_NAME: "dve-simple-py"
REGISTRY: "registry.gitlab.com"

# Test stage (runs in parallel for all images)
test:python38:
  stage: test
  image: python:3.8
  script:
    - uv pip install -r requirements.txt
    - pytest
  rules:
    - if: $CI_COMMIT_BRANCH == "development" || $CI_COMMIT_BRANCH == "main"

# Build stage (parallel jobs for each image)
build:python38:
  stage: build
  script:
    - podman build -t $REGISTRY/$IMAGE_NAME:python38-$CI_COMMIT_SHORT_SHA .
    - podman push $REGISTRY/$IMAGE_NAME:python38-$CI_COMMIT_SHORT_SHA
  needs: ["test:python38"] # Only run if tests pass
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

build:python39:
  stage: build
  script:
    - podman build -t $REGISTRY/$IMAGE_NAME:python39-$CI_COMMIT_SHORT_SHA .
    - podman push $REGISTRY/$IMAGE_NAME:python39-$CI_COMMIT_SHORT_SHA
  needs: ["test:python39"]
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

# Deploy stage (depends on all build jobs)
deploy:production:
  stage: deploy
  script:
    - echo "Deploying to production"
  needs: ["build:python38", "build:python39"] # Only run if all builds succeed
  rules:
    - if: $CI_COMMIT_TAG # Only run for tags

```

Failure Handling:

- If test:python38 fails, build:python38 will **not run** (due to needs).
- If build:python38 fails, deploy:production will **not run** (due to needs).
- All build jobs run in **parallel** within the build stage.

6. DevOps Strategy Synthesis

Operative Procedures

1. Configuration Management

- **Action:** Use a private GitLab repository for configuration (e.g., `dve-simple-py-config`).
- **Implementation:**
 - Clone the config repo alongside the main repo in CI/CD pipelines and local development.
 - Use CI/CD variables (`CONFIG_BRANCH`, `CONFIG_TAG`) to select the correct config version per environment.
- **Critical Points:**
 - Ensure the config repo is **private** and only accessible to authorized users/pipelines.
 - Avoid hardcoding paths; use relative paths (e.g., `../dve-simple-py-config`).

2. Environment-Specific Configuration

- **Action:** Use branches/tags in the config repo to separate environments.
- **Implementation:**
 - development → config repo development branch.
 - testing/staging → config repo testing/staging branches.
 - production → config repo **tagged releases** (e.g., `v1.0.0`).
- **Critical Points:**
 - Tag config repo releases **immutably** to ensure production stability.
 - Use CI/CD variables to dynamically set the config repo reference.

3. Inter-Repository Authorisation

- **Action:** Use **protected CI/CD variables** for tokens and **trigger tokens** for cross-repo pipelines.
- **Implementation:**
 - Store upstream/mirror tokens as **protected, masked CI/CD variables** in the downstream project.
 - Use `trigger:` in `.gitlab-ci.yml` to start upstream pipelines from downstream.
 - For GitHub mirror pushes, use a **PAT** stored as a protected variable.
- **Critical Points:**
 - **Never** store tokens in the repository (even in `.gitlab-ci.yml`).
 - Rotate tokens regularly and audit access.

4. Secrets Management

- **Action:** Centralize secrets in **protected CI/CD variables** and use `.env` files locally.
- **Implementation:**
 - **Pipeline:** Use masked CI/CD variables (e.g., `REGISTRY_TOKEN`, `DB_PASS`).
 - **Local:** Use `.env` files (excluded from Git) and load them with `python-dotenv`.
- **Critical Points:**

- Ensure `.env` files are **never committed** to Git.
- Use the same variable names in pipelines and local development for consistency.

5. Parallel Container Builds

- **Action:** Use GitLab CI/CD's `needs` keyword to create a failure-aware dependency graph.
- **Implementation:**
 - Define parallel build jobs for each container image (e.g., `build:python38`, `build:python39`).
 - Use `needs` to link `test` → `build` → `deploy` jobs.
- **Critical Points:**
 - Test jobs must run **before** build jobs (enforced via `needs`).
 - Deploy jobs must wait for **all** build jobs to succeed.

Comparison of Alternatives

Objective	Alternative 1	Alternative 2	Chosen
Configuration Management	<code>git submodule</code>	Parallel checkout	Parallel
Environment Configuration	Single branch with env-specific files	Branches/tags per environment	Branches
Inter-Repo Auth	Personal access tokens in CI variables	Project access tokens	Project
Secrets Storage	GitLab CI variables only	GitLab + local <code>.env</code> files	GitLab
Parallel Builds	Sequential jobs	Parallel jobs with <code>needs</code>	Parallel

References

- see[[^]av-1] [GitLab CI/CD Variables](#)
- see[[^]av-2] [GitLab needs Keyword](#)
- see[[^]av-3] [GitLab Trigger Pipelines](#)
- see[[^]av-4] [GitLab Protected Environments](#)
- see[[^]av-5] [Podman Login](#)
- see[[^]av-6] [Python python-dotenv](#)
- see[[^]av-7] [Git Submodules](#)

[[^]av-1] [GitLab CI/CD Variables](#) [[^]av-2] [GitLab needs Keyword](#) [[^]av-3] [GitLab Trigger Pipelines](#)
 [[^]av-4] [GitLab Protected Environments](#) [[^]av-5] [Podman Login](#) [[^]av-6] [Python python-dotenv](#)
 [[^]av-7] [Git Submodules](#)

Additional Notes

Security Considerations

- **Least Privilege:** Ensure CI/CD tokens have the **minimum required permissions** (e.g., read-only for config repo, write for mirror pushes).
- **Audit Logs:** Monitor GitLab CI/CD job logs for unauthorized access attempts.
- **Token Rotation:** Rotate all tokens (CI/CD variables, PATs) **at least every 90 days**.

Performance Optimizations

- **Cache Dependencies:** Use GitLab CI/CD [cache](#) to speed up `uv pip install` between jobs.
- **Shallow Clones:** Use `GIT_DEPTH: 1` for faster repository cloning in CI/CD.

Local Development Parity

- **Script Consistency:** Use the same scripts (e.g., `build.sh`, `test.sh`) in CI/CD and local development to ensure parity.
- **Docker/Podman Compatibility:** Test container builds locally with Podman before pushing to CI/CD.

Multi-Repository Workflow

- **Upstream Sync:** Regularly sync the downstream main branch with the upstream main branch to incorporate public changes.
- **Mirror Sync:** Automate mirror pushes from upstream to GitHub using a CI/CD job in the upstream project.

Python Project Automation with uv

- **Pre-Release Validation:** Run `pyright`, `ruff`, and `pytest` in a dedicated CI/CD job before building container images.
- **Post-Release:** Tag the repository and push the tag to upstream/mirror after successful validation.

Example pre-release job:

```
validate:
  stage: test
  script:
    - uv pip install pyright ruff pytest
    - pyright
    - ruff check .
    - pytest
  rules:
    - if: $CI_COMMIT_BRANCH = "main" || $CI_COMMIT_BRANCH = "development"
```

Example: Full .gitlab-ci.yml for Downstream Project

```
stages:
  - validate
  - test
  - build
  - deploy

variables:
  IMAGE_NAME: "dve-simple-py"
  REGISTRY: "registry.gitlab.com"
  CONFIG_REPO_URL: "git@gitlab.com:ub-dems/vs-base/dve-simple-py-config.git"

# Pre-release validation
validate:
  stage: validate
  script:
    - uv pip install pyright ruff pytest
    - pyright
    - ruff check .
    - pytest
  rules:
    - if: $CI_COMMIT_BRANCH = "main" || $CI_COMMIT_BRANCH = "development"

# Test stage
test:python38:
  stage: test
  image: python:3.8
  script:
    - uv pip install -r requirements.txt
    - pytest
  needs: ["validate"]
  rules:
    - if: $CI_COMMIT_BRANCH = "development" || $CI_COMMIT_BRANCH = "main"

# Build stage (parallel jobs)
build:python38:
  stage: build
  script:
    - podman build -t $REGISTRY/$IMAGE_NAME:python38-$CI_COMMIT_SHORT_SHA .
    - echo $REGISTRY_TOKEN | podman login -u $REGISTRY_USER --password-stdin $REGISTRY
    - podman push $REGISTRY/$IMAGE_NAME:python38-$CI_COMMIT_SHORT_SHA
  needs: ["test:python38"]
  rules:
    - if: $CI_COMMIT_BRANCH = "main"
```

```
build:python39:
  stage: build
  script:
    - podman build -t $REGISTRY/$IMAGE_NAME:python39-$CI_COMMIT_SHORT_SHA .
    - podman push $REGISTRY/$IMAGE_NAME:python39-$CI_COMMIT_SHORT_SHA
  needs: ["test:python39"]
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

# Deploy stage
deploy:staging:
  stage: deploy
  script:
    - echo "Deploying to staging"
  needs: ["build:python38", "build:python39"]
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
  environment:
    name: staging
    url: https://staging.example.com

deploy:production:
  stage: deploy
  script:
    - echo "Deploying to production"
  needs: ["build:python38", "build:python39"]
  rules:
    - if: $CI_COMMIT_TAG
  environment:
    name: production
    url: https://example.com

# Trigger upstream pipeline
trigger_upstream:
  stage: deploy
  trigger:
    project: ub-dems-public/ds-lab/dve-simple-py
    token: $UPSTREAM_TRIGGER_TOKEN
    strategy: depend
    variables:
      SOURCE_BRANCH: $CI_COMMIT_REF_NAME
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

```
# Push to GitHub mirror
push_to_mirror:
  stage: deploy
  script:
    - git remote add mirror https://oauth2:$GITHUB_MIRROR_TOKEN@github.com/ub-dems/dve-simple-py
    - git push mirror $CI_COMMIT_REF_NAME
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Here's a comprehensive, structured strategy for your **GitLab CI/CD multi-repository setup** for the Python project managed with `uv`. The canvas above contains the full, detailed response—you can **edit, expand, or iterate** on it directly.

Key Takeaways

1. Configuration Management

- **Recommendation:** Use a **parallel checkout** of a private configuration repository (accessed via relative path).
- **Why:** Better privacy, simplicity, and CI/CD integration compared to `git submodule`.

2. Environment-Specific Configuration

- Use **branches/tags** in the config repo to separate environments (e.g., development, staging, production).
- Resolve config URLs dynamically using **CI/CD variables** (`CONFIG_BRANCH`, `CONFIG_TAG`).

3. Inter-Repository Authorisation

- Use **protected CI/CD variables** for tokens (e.g., `UPSTREAM_TRIGGER_TOKEN`, `GITHUB_MIRROR_TOKEN`).
- Override defaults at pipeline trigger time via **GitLab UI/API** or **trigger tokens**.
- For protected branches/environments, use **protected variables** and `rules/needs` in `.gitlab-ci.yml`.

4. Secrets Management

- **Pipeline:** Store secrets as **protected, masked CI/CD variables**.
- **Local:** Use `.env` files (excluded from Git) and load them with `python-dotenv`.
- Ensure `podman push` and other secret-dependent commands work identically in both contexts.

5. Parallel Container Builds

- Use GitLab's `needs` keyword to create a **failure-aware dependency graph**.
- Example: Test jobs → Build jobs (parallel) → Deploy job (depends on all builds).

6. DevOps Strategy

- **Operative Procedures:** Centralize config, use branches/tags for environments, rotate

tokens, and audit access.

- **Critical Points:** Avoid hardcoding paths, use relative paths, and ensure parity between CI/CD and local development.
-

Next Steps

- **Review the canvas** for the full details, including tables, code snippets, and references.
- **Iterate** on the `.gitlab-ci.yml` sketch or configuration strategy as needed.
- **Ask for clarifications** or adjustments to any section!

A:a

A:a - .gitlab-ci-yml Example

^

Appendix a - initial .gitlab-ci-yml GitLab pipeline

```
# =====
# .gitlab-ci.yml
#
# Shared release pipeline for the dve-simple-py codebase.
# This exact file lives in both projects:
#   - ub-dems/dve-simple-py      (private downstream fork, branches main/development)
#   - ub-dems-public/dve-simple-py (public upstream, branch main)
#
# The pipeline is tag-triggered (v*.*.*) and splits into two cooperating
# pipeline RUNS in two different projects:
#   1. Downstream validates, opens the release MR, and (on manual approval)
#      merges + tags upstream.
#   2. Creating that tag on upstream via the API starts a SECOND, independent
#      pipeline in the upstream project, which builds, pushes, mirrors, and
#      notifies.
# =====
---
spec:
  inputs:
    gd-release-op-upstream-merge:
      default: '1'
    gd-release-op-upstream-mirror:
      default: '1'
    gd-release-op-upstream-image:
      default: '0'
    gd-release-op-upstream-notify:
      default: '0'
    gd-downstream-project:
      default: "ub-dems/vs-base/dve-simple-py"
    gd-upstream-project:
      default: "ub-dems-public/ds-labs/dve-simple-py"
```

```

gd-upstream-project-enc:
  default: "ub-dems-public%2Fds-labs%2Fdve-simple-py"
gd-downstream-project-head:
  default: "develop"
gd-downstream-project-main:
  default: "main"
gd-upstream-project-main:
  default: "main"
gd-version-schema:
  default: '^\\d\\.\\d+(\\.\\d+)?$'
gd-python-version:
  default: '3.14'
environment:
  options:
    - development
    - staging
    - production
  default: production
---

stages:
  - lint
  - test
  - release
  - publish

default:
  image: python:${[ inputs.gd-python-version ]}-slim

variables:

  # Non-sensitive defaults only. REGISTRY_URL, REGISTRY_USER, REGISTRY_PASSWORD,
  # UPSTREAM_TOKEN, GITHUB_MIRROR_URL, NOTIFY_EMAILS, SMTP_*, SLACK_WEBHOOK_URL
  # are pre-configured project CI/CD variables and are intentionally NOT
  # redeclared here.
  DOWNSTREAM_PROJECT_PATH: "${[ inputs.gd-downstream-project ]}"
  UPSTREAM_PROJECT_PATH: "${[ inputs.gd-upstream-project ]}"
  UPSTREAM_PROJECT_PATH_ENCODED: "${[ inputs.gd-upstream-project-enc ]}"
  MR_SOURCE_BRANCH: "${[ inputs.gd-downstream-project-main ]}"
  MR_TARGET_BRANCH: "${[ inputs.gd-upstream-project-main ]}"
  GIT_DEPTH: "0"

  # -----
  # Disk-budget notes

```

```

# -----
# GitLab SaaS shared runners carry a 20 GB working-directory quota.
# The CUDA-bundled torch wheel + triton alone exceed this budget.
#
# Fix: redirect the "torch-gpu" named index to the CPU wheel server for all
# CI jobs via UV_INDEX_TORCH_GPU. Downstream consumers running on GPU hosts
# omit this variable and continue to receive CUDA wheels from the cu130 index.
#
# Cache strategy: persist only $UV_CACHE_DIR (downloaded wheel archives).
# Do NOT cache .venv – extracting from the wheel cache is fast (~30 s) and
# avoids the cache-serialisation phase that itself hits the disk quota.
# -----

# --- uv installation ---
UV_VERSION: "latest"

# --- uv cache location (project-relative, included in GitLab cache) ---
UV_CACHE_DIR: "$CI_PROJECT_DIR/.cache/uv"

# --- index override: redirect "torch-gpu" to CPU wheel server in CI ---
# uv resolves UV_INDEX_<NAME_UPPER> where hyphens become underscores.
# CPU wheels do not include triton, saving ~1.5 GB per job.
UV_INDEX_TORCH_GPU: "https://download.pytorch.org/whl/cpu"

# --- disable progress bars (cleaner CI logs) ---
UV_NO_PROGRESS: "1"

# --- never use the system Python; always use uv-managed interpreter ---
UV_PYTHON_PREFERENCE: "only-managed"

```

cache:

```

policy: pull-push
# key:
#   files:
#     - pyproject.toml
#     - uv.lock
# paths:
#   - .cache/uv

```

workflow:

```

rules:
  - if: $CI_COMMIT_MESSAGE =~ /^WIP/
    when: never
  - if: $CI_COMMIT_TAG

```

```

- if: $CI_COMMIT_BRANCH = $CI_DEFAULT_BRANCH
- if: $CI_COMMIT_REF_PROTECTED = "true"

# -----
# Context Rules
# -----

.ctx-develop:
  rules:
    - if: '$CI_PROJECT_PATH = "$[[ inputs.gd-downstream-project ]]"'
    - if: '$CI_COMMIT_BRANCH = "$[[ inputs.gd-downstream-project-head ]]"'

.ctx-release-tag:
  rules:
    - if: '"$[[ inputs.gd-release-op-upstream-merge ]]" = "1"'
    - if: '$CI_COMMIT_TAG =~ /$[[ inputs.gd-version-schema ]]/'

.ctx-release-downstream:
  extends:
    - .ctx-release-tag
  rules:
    - if: '$CI_PROJECT_PATH = "$[[ inputs.gd-downstream-project ]]"'
    - if: '$CI_COMMIT_BRANCH = "$[[ inputs.gd-downstream-project-main ]]"'

.ctx-release-upstream:
  extends:
    - .ctx-release-tag
  rules:
    - if: '$CI_PROJECT_PATH = "$[[ inputs.gd-upstream-project ]]"'
    - if: '$CI_COMMIT_BRANCH = "$[[ inputs.gd-upstream-project-main ]]"'

.ctx-release-mirror:
  extends:
    - .ctx-release-upstream
  rules:
    - if: '"$[[ inputs.gd-release-op-upstream-mirror ]]" = "1"'

.ctx-release-image:
  extends:
    - .ctx-release-upstream
  rules:
    - if: '"$[[ inputs.gd-release-op-upstream-image ]]" = "1"'

.ctx-release-notify:
  extends:

```

```

    - .ctx-release-upstream
rules:
  - if: '$[[ inputs.gd-release-op-upstream-notify ]]' = "1"'

# -----
# Base Images
# -----

.gitlab-api-base:
  image: alpine:3.20
  variables:
    GIT_STRATEGY: none
  before_script:
    - apk add --no-cache curl jq

.git-base:
  image: alpine/git:2.45.2
  before_script:
    - apk add --no-cache curl jq

.notify-base:
  extends:
    - .gitlab-api-base
  before_script:
    - apk add --no-cache msmtplib

.podman-base:
  image: quay.io/podman/stable:v5

.uv-validate-base:
  image: python:${[[ inputs.gd-python-version ]]}-slim
  variables:
    UV_CACHE_DIR: "${CI_PROJECT_DIR}/.cache/uv"
  cache:
    # Only the uv download cache is persisted. .venv is intentionally excluded.
    key: "uv-${CI_COMMIT_REF_SLUG}"
    # files:
    #   - pyproject.toml
    #   - uv.lock
    paths:
      - .cache/uv
  before_script:
    - echo "#__uv:init__"
    - env

```

```

- df -h
# - apt-get update -qq
# - apt-get install -y --no-install-recommends ca-certificates curl git
# - rm -rf /var/lib/apt/lists/*
# - curl -LsSf https://astral.sh/uv/install.sh | sh
# - export PATH="$HOME/.local/bin:$PATH"
- pip install --quiet uv
# - uv sync --frozen

```

.uv-validate-lint:**extends:**

```
- .uv-validate-base
```

before_script:

```

# Lint jobs operate on source text only.
# --only-group dev : install ruff and its peers; skip main dependencies.
# --no-install-project : do not install the project package itself.
# Result: no torch, no torchvision, no triton, minimal disk use (~150 MB).
- uv sync --only-group dev --no-install-project
- uv tree --show-sizes
- uv pip list
- du -sh .venv
- df -h

```

.uv-validate-check:**extends:**

```
- .uv-validate-base
```

before_script:

```

# pyright requires all project imports to be resolvable, so main deps
# (including CPU torch) must be present. UV_INDEX_TORCH_GPU redirects
# the cu130 index to the CPU wheel server; triton is not a dep of CPU torch.
- uv sync --group dev
- df -h

```

.uv-validate-test:**extends:**

```
- .uv-validate-base
```

before_script:

```

# pyright requires all project imports to be resolvable, so main deps
# (including CPU torch) must be present. UV_INDEX_TORCH_GPU redirects
# the cu130 index to the CPU wheel server; triton is not a dep of CPU torch.
- uv sync --group dev

```

```

# =====
# Pipeline: VALIDATION: downstream head only
# -----

```

```

# - after feature branch merge-request commit
# - before head → main MR (blocking)
# =====

# -----
# Stage: lint (downstream head only)
# -----

linting:
  stage: lint
  extends:
    - .ctx-develop
    - .uv-validate-lint
  script:
    - uv run ruff check

formatting:
  stage: lint
  extends:
    - .ctx-develop
    - .uv-validate-lint
  script:
    - uv run ruff format --check

typing:
  stage: lint
  extends:
    - .ctx-develop
    - .uv-validate-check
  script:
    - uv run pyright

licensing:
  stage: lint
  extends:
    - .ctx-develop
    - .uv-validate-check
  script:
    - uv run reuse lint

# -----
# Stage: test (downstream head only)
# -----

```



```

testing:
  stage: test
  extends:
    - .ctx-develop
    - .uv-validate-test
  script:
    - uv run pytest -v

# =====
# Pipeline: RELEASE: downstream main + upstream main
# -----
# - after downstream main annotated version tag commit
# - forward downstream main → upstream main MR
# =====

# -----
# Stage: release #validation (downstream only)
# -----

release-checks:
  stage: release
  extends:
    - .ctx-release-downstream
    - .uv-validate-check
  script:
    - uv run ruff check
    - uv run ruff format --check
    - uv run pyright
    - uv run reuse lint

release-tests:
  stage: release
  extends:
    - .ctx-release-downstream
    - .uv-validate-test
  script:
    - uv run pytest -v
    # - uv run pytest -v --cov

# -----
# Stage: release #merge-request (downstream only) - upstream MR create
# -----

# Project scope      : downstream only

```

```

# Trigger condition: tag *.*.* (workflow:rules) AND downstream project path
# Required vars      : UPSTREAM_TOKEN

create-merge-request:
  extends:
    - .ctx-release-downstream
    - .gitlab-api-base
  stage: release
  dependencies:
    - release-checks
    - release-tests
  script:
    - |
      UPSTREAM_PROJECT_ID=$(curl --silent --show-error --fail \
        --header "PRIVATE-TOKEN: ${UPSTREAM_TOKEN}" \
        "${CI_API_V4_URL}/projects/${UPSTREAM_PROJECT_PATH_ENCODED}" | jq -r .id)

      RESPONSE=$(curl --silent --show-error --fail \
        --request POST \
        --header "PRIVATE-TOKEN: ${UPSTREAM_TOKEN}" \
        --data-urlencode "source_branch=${MR_SOURCE_BRANCH}" \
        --data-urlencode "target_branch=${MR_TARGET_BRANCH}" \
        --data-urlencode "target_project_id=${UPSTREAM_PROJECT_ID}" \
        --data-urlencode "title=Release ${CI_COMMIT_TAG}: sync downstream to upstream main" \
        --data-urlencode "description=Automated release MR for tag ${CI_COMMIT_TAG}. Source: ${CI_COMMIT_SHA}" \
        "${CI_API_V4_URL}/projects/${CI_PROJECT_ID}/merge_requests")

      MR_IID=$(echo "${RESPONSE}" | jq -r '.iid // empty')
      if [ -z "${MR_IID}" ]; then
        echo "Failed to create merge request:"
        echo "${RESPONSE}" | jq .
        exit 1
      fi

      echo "Created MR !${MR_IID} on upstream project ${UPSTREAM_PROJECT_ID}"
      echo "MR_IID=${MR_IID}" >> mr.env
      echo "UPSTREAM_PROJECT_ID=${UPSTREAM_PROJECT_ID}" >> mr.env
  artifacts:
    reports:
      dotenv: mr.env

# -----
# Stage: release #merge-request (downstream only) - upstream MR merge + tag
# -----

# Project scope      : downstream only (ub-dems/dve-simple-py)

```

```

# Trigger condition: tag v*.*.* (workflow:rules) AND downstream project path;
#                               MANUAL (see "Stage 3 approval gate" above)
# Required vars      : UPSTREAM_TOKEN
trigger-upstream-merge-and-tag:
  extends:
    - .ctx-release-downstream
    - .gitlab-api-base
  stage: release
  needs:
    - job: create-merge-request
      artifacts: true
  dependencies:
    - create-merge-request
  script:
    - |
      echo "Waiting for the upstream validation pipeline on MR !${MR_IID} ..."
      PIPELINE_STATUS="none"
      for i in $(seq 1 60); do
        MR_JSON=$(curl --silent --show-error --fail \
          --header "PRIVATE-TOKEN: ${UPSTREAM_TOKEN}" \
          "${CI_API_V4_URL}/projects/${UPSTREAM_PROJECT_ID}/merge_requests/${MR_IID}")
        PIPELINE_STATUS=$(echo "${MR_JSON}" | jq -r '.head_pipeline.status // "none"')
        echo "  upstream pipeline status: ${PIPELINE_STATUS}"
        [ "${PIPELINE_STATUS}" = "success" ] && break
        if [ "${PIPELINE_STATUS}" = "failed" ] || [ "${PIPELINE_STATUS}" = "canceled" ]; then
          echo "Upstream validation pipeline did not succeed. Aborting."
          exit 1
        fi
        sleep 30
      done
      if [ "${PIPELINE_STATUS}" ≠ "success" ]; then
        echo "Timed out waiting for the upstream validation pipeline."
        exit 1
      fi

      curl --silent --show-error --fail \
        --request PUT \
        --header "PRIVATE-TOKEN: ${UPSTREAM_TOKEN}" \
        --data-urlencode "should_remove_source_branch=false" \
        --data-urlencode "merge_commit_message=Release ${CI_COMMIT_TAG}" \
        "${CI_API_V4_URL}/projects/${UPSTREAM_PROJECT_ID}/merge_requests/${MR_IID}/merge"

      echo "Waiting for the merge to complete ..."
      MR_STATE="unknown"
      for i in $(seq 1 20); do

```

```

MR_JSON=$(curl --silent --show-error --fail \
  --header "PRIVATE-TOKEN: ${UPSTREAM_TOKEN}" \
  "${CI_API_V4_URL}/projects/${UPSTREAM_PROJECT_ID}/merge_requests/${MR_IID}")
MR_STATE=$(echo "${MR_JSON}" | jq -r .state)
if [ "${MR_STATE}" = "merged" ]; then
  MERGE_COMMIT_SHA=$(echo "${MR_JSON}" | jq -r .merge_commit_sha)
  break
fi
sleep 15
done
if [ "${MR_STATE}" ≠ "merged" ]; then
  echo "Merge request was not merged in time."
  exit 1
fi

echo "Tagging upstream main at ${MERGE_COMMIT_SHA} as ${CI_COMMIT_TAG}"
curl --silent --show-error --fail \
  --request POST \
  --header "PRIVATE-TOKEN: ${UPSTREAM_TOKEN}" \
  --data-urlencode "tag_name=${CI_COMMIT_TAG}" \
  --data-urlencode "ref=${MERGE_COMMIT_SHA}" \
  --data-urlencode "message=Release ${CI_COMMIT_TAG}" \
  "${CI_API_V4_URL}/projects/${UPSTREAM_PROJECT_ID}/repository/tags"

# -----
# Stage: mirror-push (upstream only)
# -----

# Project scope      : upstream only
# Trigger condition: tag *.*.* (workflow:rules) AND upstream project path;
#                      runs only if enabled
# Required vars      : GITHUB_MIRROR_URL
mirror-push:
  stage: release
  extends:
    - .ctx-release-mirror
    - .git-base
  dependencies: []
  variables:
    GIT_STRATEGY: clone
    GIT_DEPTH: "0"
  script:
    - git remote add mirror "${GITHUB_MIRROR_URL}" || git remote set-url mirror "${GITHUB_MIRROR_URL}"
    - git push --force-with-lease mirror "${CI_COMMIT_TAG}"
    - git push --force-with-lease mirror "HEAD:refs/heads/main"

```

```

# -----
# Stage: release #publish (unstream only) - podman build + save
# -----

# Project scope      : upstream only
# Trigger condition: tag *.*.* (workflow:rules) AND upstream project path;
#                   this run starts because the tag above was created on the
#                   upstream project via the API in the previous stage
# Required vars      : none
build-base-images:
  stage: release
  extends:
    - .ctx-release-image
    - .podman-base
  needs: []
  dependencies: []
  parallel:
    matrix:
      # Placeholder identifiers – replace with the real image names produced
      # by scripts/build-images.sh (see "Additional notes" below).
      - IMAGE_NAME: ["anchor", "cuda", "base", "runtime"]
  script:
    - build.sh image "${IMAGE_NAME}"
    - mkdir -p images
    - podman save -o "images/${IMAGE_NAME}.tar" "localhost/${IMAGE_NAME}:${CI_COMMIT_TAG}"
  artifacts:
    paths:
      - "images/${IMAGE_NAME}.tar"
    expire_in: "1 day"

# -----
# Stage: image-push (upstream only)
# -----

# Project scope      : upstream only (ub-dems-public/dve-simple-py)
# Trigger condition: tag v*.*.* (workflow:rules) AND upstream project path;
#                   runs only if every build-image matrix job succeeded
# Required vars      : REGISTRY_URL, REGISTRY_USER, REGISTRY_PASSWORD
push-base-images:
  stage: release
  extends:
    - .ctx-release-image
    - .podman-base

```

```

needs:
  - job: build-base-images
    artifacts: true
dependencies:
  - build-base-images
script:
  - podman login -u "${REGISTRY_USER}" -p "${REGISTRY_PASSWORD}" "${REGISTRY_URL}"
  - |
    for IMAGE_NAME in anchor, cuda, base, runtime; do
      podman load -i "images/${IMAGE_NAME}.tar"
      podman push "localhost/${IMAGE_NAME}:${CI_COMMIT_TAG}" \
        "${REGISTRY_URL}/${IMAGE_NAME}:${CI_COMMIT_TAG}"
    done

# -----
# Stage: release #notify (upstream only) - email + slack
# -----

# Project scope      : upstream only (ub-dems-public/dve-simple-py)
# Trigger condition: tag v*.*.* (workflow:rules) AND upstream project path;
#                   runs only if mirror-push succeeded
# Required vars      : NOTIFY_EMAILS, SMTP_HOST, SMTP_USER, SMTP_PASSWORD
release-notify-email:
  stage: release
  extends:
    - .ctx-release-notify
    - .notify-base
  dependencies: []
  variables:
    GIT_STRATEGY: none
  script:
    - |
      cat > /tmp/msmtprc <<-EOF
      account default
      host ${SMTP_HOST}
      port 587
      auth on
      user ${SMTP_USER}
      password ${SMTP_PASSWORD}
      tls on
      from ${SMTP_USER}
      EOF
      chmod 600 /tmp/msmtprc

      IFS=',' read -ra RECIPIENTS <<< "${NOTIFY_EMAILS}"

```

```

    for RECIPIENT in "${RECIPIENTS[@]}"; do
        printf "Subject: Release %s published\n\nRelease %s of %s has been merged, tagged, built\n\n"
            "${CI_COMMIT_TAG}" "${CI_COMMIT_TAG}" "${CI_PROJECT_PATH}" \
            | msmtprc -a default --file=/tmp/msmtprc "${RECIPIENT}"
    done

# Project scope      : upstream only
# Trigger condition: tag v*.*.* (workflow:rules) AND upstream project path;
#                   runs only if mirror-push succeeded
# Required vars      : SLACK_WEBHOOK_URL
release-notify-slack:
  stage: release
  extends:
    - .ctx-release-notify
    - .notify-base
  dependencies: []
  variables:
    GIT_STRATEGY: none
  script:
    - |
      curl --silent --show-error --fail \
        --request POST \
        --header "Content-Type: application/json" \
        --data "{\"text\": \"Release ${CI_COMMIT_TAG} published for ${CI_PROJECT_PATH}.\\\"}\" \
        \"${SLACK_WEBHOOK_URL}\"

```